

Pour les valeurs qui appartiennent à **translatable_string_type**, la contrainte s'applique à la chaîne qui est dans la langue source dans laquelle le domaine de la propriété était défini. Cette langue source peut être définie dans l'attribut **source_language** de **administrative_data** de la propriété. Si cet attribut n'existe pas, cette langue source est supposée être connue de l'utilisateur du dictionnaire.

EXPRESS specification:

```

*)
ENTITY string_size_constraint
SUBTYPE OF (domain_constraint);
    min_length: OPTIONAL INTEGER;
    max_length: OPTIONAL INTEGER;
WHERE
    WR1: (min_length >= 0) AND (max_length >= min_length);
END_ENTITY; -- string_size_constraint
(*

```

Définitions des attributs:

min_length: la longueur minimale pour les chaînes qui sont admissibles comme valeurs pour la propriété identifiée par la propriété **constrained_property**.

max_length: la longueur maximale pour les chaînes qui sont admissibles comme valeurs pour la propriété identifiée par la propriété **constrained_property**.

NOTE 3 Si la valeur **min_length** n'existe pas, 0 est sous-entendu. Si la valeur **max_length** n'existe pas, "non borné" est sous-entendu.

Propositions formelles:

WR1: **min_length** et **max_length** définissent les bornes correctes.

7.3.15 String_pattern_constraint

Une entité **string_pattern_constraint** limite le domaine des valeurs d'un **string_type**, ou de n'importe quel de ses sous-types, à des valeurs de chaîne qui correspondent à un profil particulier de caractères génériques ("pattern"). La syntaxe **pattern** est définie par une expression rationnelle XML et les algorithmes de concordance associés qui sont définis par le Schéma XML Partie 2: Recommandation pour les datatypes.

NOTE 1 Un domaine de valeurs de propriétés **string_type** est soit un **string_type**, soit un **non_translatable_string_type**, soit un **translatable_string_type**, soit un **URI_type**, soit un **non_quantitative_code_type**, soit un **date_data_type**, soit un **time_data_type**, soit un **date_time_data_type**.

Dans le cas de **string_type**, de **non_translatable_string_type**, de **URI_type**, de **non_quantitative_code_type**, de **date_data_type**, de **time_data_type** ou de **date_time_data_type**, la contrainte s'applique à la chaîne (unique) qui est la valeur du type de données. Pour **non_quantitative_code_type**, la contrainte s'applique au code.

Pour les valeurs qui appartiennent à des entités **translatable_string_type**, la contrainte s'applique à la chaîne qui est dans la langue source dans laquelle le domaine de la propriété était défini. Cette langue source peut être définie dans l'attribut **source_language** de l'entité **administrative_data** de la propriété; si cet attribut n'existe pas, cette langue source est supposée être connue de l'utilisateur du dictionnaire.

NOTE 2 Dans le cas de **non_quantitative_code_type**, de **date_data_type**, de **time_data_type** ou de **date_time_data_type**, il convient que le **pattern** se conforme aux propositions informelles définies dans les types de données correspondants.

EXPRESS specification:

```

*)
ENTITY string_pattern_constraint
SUBTYPE OF (domain_constraint);
    pattern: STRING;
END_ENTITY; -- string_pattern_constraint
( *

```

Définition d'attributs:

pattern: le profil des valeurs de chaîne qui sont admissibles comme valeurs de la propriété identifiée par la propriété contrainte.

Proposition informelle:

IP1: la syntaxe **pattern** doit se conformer à la syntaxe d'expression rationnelle XML et aux algorithmes de concordance associés qui sont définis par le Schéma XML Partie 2: Recommandation pour les datatypes.

EXEMPLE Le profil du Schéma XML qui correspond à l'expression SQL SIMILAR "[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]" est "[0-9]{4}\-[0-9]{2}\-[0-9]{2}". Il permet de faire concorder des chaînes à "2009-05-31".

7.3.16 Cardinality_constraint

Une entité **cardinality_constraint** limite la cardinalité d'un type de données agrégé.

NOTE 1 La plage de cardinalités qui en résulte est l'intersection des plages de cardinalités préexistantes et de celle définie par **cardinality_constraint**.

NOTE 2 Les contraintes **cardinality_constraint** sont interdites sur le type **array_type**.

EXPRESS specification:

```

*)
ENTITY cardinality_constraint
SUBTYPE OF (domain_constraint);
    bound_1: OPTIONAL INTEGER;
    bound_2: OPTIONAL INTEGER;
WHERE
    WR1: (bound_1 >= 0) AND (bound_2 >= bound_1);
END_ENTITY; -- cardinality_constraint
( *

```

Définitions des attributs:

bound_1: la borne inférieure de la cardinalité.

bound_2: la borne supérieure de la cardinalité.

NOTE 3 Si **bound_1** n'existe pas, la cardinalité minimale est 0. Lorsque **bound_2** n'existe pas, il n'y a aucune contrainte sur la cardinalité maximale.

Propositions formelles:

WR1: **bound_1** et **bound_2** définissent les bornes correctes.

7.4 Définitions des types de l'ISO13584_IEC61360_class_constraint_schema

7.4.1 Généralités

Le présent paragraphe définit le type dans l'ISO13584_IEC61360_class_constraint_schema.

7.4.2 Constraint_or_constraint_id

Le `constraint_or_constraint_id` est soit une `constraint` (contrainte), soit un `constraint_identifier` (identificateur de contrainte).

EXPRESS specification:

```
*)
TYPE constraint_or_constraint_id =
    SELECT (constraint, constraint_identifier);
END_TYPE; -- constraint_or_constraint_id
(*
```

7.5 Définition de fonctions de l'ISO13584_IEC61360_class_constraint_schema

7.5.1 Généralités

Le présent paragraphe définit les fonctions dans l'ISO13584_IEC61360_class_constraint_schema.

7.5.2 Fonction integer_values_in_range

La fonction `integer_values_in_range` calcule les valeurs entières qui appartiennent à une plage entière définie par sa borne inférieure et sa borne supérieure. Elle retourne "indéterminé" (?) lorsque l'une ou l'autre bornes est indéterminée.

EXPRESS specification:

```
*)
FUNCTION integer_values_in_range(
    low_bound, high_bound: INTEGER): SET OF INTEGER;
LOCAL
    i: INTEGER;
    result : SET OF INTEGER:= [];
END_LOCAL;
IF EXISTS (low_bound) AND EXISTS (high_bound)
THEN
    REPEAT i := low_bound TO high_bound;
        result := result + [i];
    END_REPEAT;
    RETURN(result);
ELSE
    RETURN(?);
END_IF;
END_FUNCTION; -- integer_values_in_range
(*
```

7.5.3 Fonction correct_precondition

La fonction `correct_precondition` s'assure que la précondition de l'entité `configuration_control_constraint` définie par `cons` utilise seulement des propriétés qui sont

applicables à la classe **cl**. Elle retourne un résultat logique qui est UNKNOWN lorsque l'ensemble complet des propriétés applicables de la classe ne peut pas être calculé.

EXPRESS specification:

```

*)
FUNCTION correct_precondition(
    cons: configuration_control_constraint; cl:class): LOGICAL;
LOCAL
    prop: SET OF property_BSU:= [];
END_LOCAL;
REPEAT i := 1 to SIZEOF (cons.precondition);
    prop := prop + cons.precondition[i].referenced_property;
END_REPEAT;

IF prop <= cl.known_applicable_properties
THEN RETURN (TRUE);
ELSE
    IF all_class_descriptions_reachable(cl.identified_by)
    THEN RETURN (FALSE);
    ELSE RETURN (UNKNOWN);
    END_IF;
END_IF;
END_FUNCTION; -- correct_precondition
(
```

7.5.4 Fonction correct_constraint_type

La fonction **correct_constraint_type** s'assure que le **domain_constraint** défini par **cons** est compatible avec **data_type** définie par **typ**. Elle retourne un élément logique qui est UNKNOWN lorsque **domain_constraint** définie par **cons** n'est pas l'un des sous-types définis dans le schéma **ISO13584_IEC61360_class_constraint_schema**.

EXPRESS specification:

```

*)
FUNCTION correct_constraint_type(
    cons: domain_constraint; typ:data_type): LOGICAL;

(*cas de contrainte de sous-classe*)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    + 'SUBCLASS_CONSTRAINT') IN TYPEOF(cons)
THEN
    (*le type de données doit être class_reference_type*)
    IF NOT
        ('ISO13584_IEC61360_DICTIONARY_SCHEMA.CLASS_REFERENCE_TYPE'
            IN TYPEOF (typ))
    THEN RETURN(FALSE);
    END_IF;

    (*les cons.subclasses doivent consister en sous-classes pour
la classe
    qui a défini le domaine initial de typ.*)
    IF NOT (QUERY (sc <* cons.subclasses |

```

```

        definition_available_implies
        (sc,definition_available_implies
        (typ\class_reference_type.domain,is_subclass(sc.definition
n[1]
        ,      typ\class_reference_type.domain.definition[1])))=
        false)
        = [])
    THEN RETURN (FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*cas de contrainte de sous-type d'entité*)

IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'ENTITY_SUBTYPE_CONSTRAINT') IN TYPEOF (CONS))
THEN

    (* le type de données est class_reference_type*)
    IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.ENTITY_INSTANCE_TYPE') IN TYPEOF (typ))
    THEN RETURN (FALSE);
    END_IF;

    (* le subtype_name doit définir un sous-type pour l'entité
    entity_instance_type du constraint *)
    IF NOT (cons\entity_subtype_constraint.subtype_names
        >= typ\entity_instance_type.type_name)
    THEN RETURN (FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*cas d'enumeration_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.ENUMERATION_CONSTRAINT') IN TYPEOF (CONS)
THEN

    (* toutes les valeurs appartenant au sous-ensemble de valeurs
    doivent être compatibles avec le type de données typ. *)
    IF (QUERY (val<*cons.subset |
        NOT compatible_data_type_and_value ( typ, val))<> [])
    THEN RETURN (FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*cas de range_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA.RANGE_CONSTRAINT'
    IN TYPEOF (CONS))

```

THEN

(*si le data type est un integer_type, alors min_value et max_value doivent être des INTEGER.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.INTEGER_TYPE'
    IN TYPEOF (typ)) AND
    NOT ('INTEGER' IN TYPEOF (cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*si le type de données est un rational_type, min_value et max_value doivent être rationnelles.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
    IN TYPEOF (typ)) AND
    NOT ('ISO13584_INSTANCE_RESOURCE_SCHEMA.RATIONAL_VALUE'
    IN TYPEOF (cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*si le type de données est un real_type, min_value et max_value doivent être REAL.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
    IN TYPEOF (typ)) AND NOT ('REAL' IN TYPEOF
(cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*toutes les valeurs de la plage doivent appartenir aux valeurs admissibles définies par le type.*)

```
IF (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    + '.NON_QUANTITATIVE_INT_TYPE') IN TYPEOF (typ))
AND NOT
    (integer_values_in_range(cons.min_value, cons.max_value)
    <= allowed_values_integer_types (typ))
THEN RETURN(FALSE);
END_IF;
```

```
RETURN (TRUE);
END_IF;
```

(*cas d'entité string_size_constraint*)

```
IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    + '.STRING_SIZE_CONSTRAINT') IN TYPEOF (CONS)
THEN
```

(* le type de données doit être un string_type ou n'importe lequel de ses sous-types. *)

```
IF NOT ('ISO13584_IEC61360_DICTIONARY_SCHEMA.STRING_TYPE'
    IN TYPEOF (typ))
THEN RETURN(FALSE);
END_IF;
```

```
RETURN (TRUE);
END_IF;
```

```

(*cas d'entité string_pattern_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.STRING_PATTERN_CONSTRAINT') IN TYPEOF (CONS)
THEN

    (* le type de données doit être un string_type ou n'importe lequel
    de ses sous-types. *)
    IF NOT ('ISO13584_IEC61360_DICTIONARY_SCHEMA.STRING_TYPE'
        IN TYPEOF (typ))
    THEN RETURN(FALSE);
    END_IF;
RETURN (TRUE);
END_IF;

(*cas d'entité cardinality_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.CARDINALITY_CONSTRAINT') IN TYPEOF (CONS)
THEN

    (* le type de données doit être un type agrégé mais pas une matrice
    array*)
    IF (NOT(
        ('ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
        + '.ENTITY_INSTANCE_TYPE_FOR_AGGREGATE')
        IN TYPEOF(typ)))
    THEN
        RETURN(FALSE);
    END_IF;

    IF ('ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
        + '.ARRAY_TYPE' IN TYPEOF(typ.type_structure))
    THEN
        RETURN(FALSE);
    END_IF;
    RETURN (TRUE);
END_IF;

RETURN (UNKNOWN);

END_FUNCTION; -- correct_constraint_type
(*

```

7.5.5 Fonction compatible_data_type_and_value

La fonction **compatible_data_type_and_value** vérifie si une valeur **val** d'une **primitive_value** est de type compatible avec le type défini par un type **dom**. Elle retourne un LOGICAL qui est TRUE lorsqu'ils sont compatibles et FALSE lorsqu'ils ne le sont pas. Cette fonction retourne UNKNOWN si le type de données de **val** est un **uncontrolled_instance_value** (voir ISO 13584-24:2003) ou lorsque le type n'est pas l'un des types définis dans le schéma **ISO13584_instance_resource_schema**.

NOTE La valeur de **val** peut ou peut ne pas exister.

EXPRESS specification:

```

*)
FUNCTION compatible_data_type_and_value(dom: data_type;
    val: primitive_value): LOGICAL;

LOCAL
    temp: class_BSU;
    set_string: SET OF STRING := [];
    set_integer: SET OF INTEGER := [];
    code_type: non_quantitative_code_type;
    int_type: non_quantitative_int_type;
END_LOCAL;

(* les déclarations express suivantes traitent des types simples.
*)

IF      ('ISO13584_INSTANCE_RESOURCE_SCHEMA.INTEGER_VALUE'      IN
TYPEOF(val))
THEN
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.' +
        'NON_QUANTITATIVE_INT_TYPE' IN TYPEOF (dom))
    THEN
        set_integer := [];
        int_type := dom;
        REPEAT j := 1 TO SIZEOF(int_type.domain.its_values);
            set_integer := set_integer +
                int_type.domain.its_values[j].value_code;
        END_REPEAT;

        RETURN(val IN set_integer);

    ELSE
        RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
            IN TYPEOF (dom)) OR
            (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
            IN TYPEOF (dom))
            AND
            NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
            IN TYPEOF (dom))
            OR
            ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
            IN TYPEOF (dom)))));

    END_IF;
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.REAL_VALUE' IN TYPEOF(val))
THEN
    RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
        IN TYPEOF (dom)) OR
        (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
        IN TYPEOF (dom))
        AND NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
        IN TYPEOF (dom))

```



```

        OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
            IN TYPEOF (dom))));
END_IF;

IF      ('ISO13584_INSTANCE_RESOURCE_SCHEMA.RATIONAL_VALUE'      IN
TYPEOF(val))
THEN
    RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL _TYPE'
        IN TYPEOF (dom)) OR
        (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
        IN TYPEOF (dom))
        AND NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
        IN TYPEOF (dom))
        OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
            IN TYPEOF (dom))));
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.STRING_VALUE'
    IN TYPEOF(val))
THEN
    IF (('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
        '.NON_QUANTITATIVE_CODE_TYPE') IN TYPEOF (dom))
    THEN
        set_string := [];
        code_type := dom;
        REPEAT j := 1 TO SIZEOF(code_type.domain.its_values);
            set_string := set_string +
                code_type.domain.its_values[j].value_code;
        END_REPEAT;

        RETURN(val IN set_string);

    ELSE
        RETURN('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.STRING_TYPE' IN TYPEOF (dom));
    END_IF;
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.TRANSLATED_STRING_VALUE'
    IN TYPEOF(val))
THEN
    RETURN('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
        '.TRANSLATABLE_STRING_TYPE' IN TYPEOF (dom));
END_IF;

(* les déclarations express suivantes traitent des types complexes.
*)

IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA.DIC_CLASS_INSTANCE'
    IN TYPEOF(val)
THEN
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.CLASS_REFERENCE_TYPE'
        IN TYPEOF (dom))
    THEN

```

```

        temp := dom.domain;
        RETURN(compatible_class_and_class(temp,
            val\dic_class_instance.class_def));
    ELSE
        RETURN(FALSE);
    END_IF;
END_IF;

IF      'ISO13584_INSTANCE_RESOURCE_SCHEMA.LEVEL_SPEC_VALUE'      IN
TYPEOF(val) THEN
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.LEVEL_TYPE'
        IN TYPEOF (dom))
    THEN
        RETURN(compatible_level_type_and_instance(
            dom.levels,
            TYPEOF(dom.value_type),
            val));
    ELSE
        RETURN(FALSE);
    END_IF;
END_IF;

(* les déclarations express suivantes traitent des types agrégés.*)

IF
'ISO13584_AGGREGATE_VALUE_SCHEMA.AGGREGATE_ENTITY_INSTANCE_VALUE'
IN TYPEOF(val) THEN

    IF (NOT(
        'ISO13584_IEC61360_DICTIONARY_SCHEMA.ENTITY_INSTANCE_TYPE'
        IN TYPEOF(dom)))
    THEN
        RETURN(FALSE);
    END_IF;

    IF (NOT(
        'ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
        + '.AGGREGATE_TYPE' IN dom.type_name))
    THEN
        RETURN(FALSE);
    END_IF;

    RETURN(compatible_aggregate_type_and_value(dom, val));

END_IF;

IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA.ENTITY_INSTANCE_VALUE'
    IN TYPEOF(val)
THEN
    IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA' +
        '.UNCONTROLLED_ENTITY_INSTANCE_VALUE'
        IN TYPEOF(val)
    THEN

```

```

        RETURN (UNKNOWN);
    END_IF;
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.ENTITY_INSTANCE_TYPE'
        IN TYPEOF (dom))
        AND (dom.type_name <= TYPEOF(val))
    THEN
        RETURN (TRUE);
    ELSE
        RETURN (FALSE);
    END_IF;
END_IF;

RETURN (UNKNOWN);

END_FUNCTION; -- compatible_data_type_and_value
(*)

```

7.6 Définition de règles de l'ISO13584_IEC61360_class_constraint_schema

7.6.1 Généralités

Le présent paragraphe définit la règle dans l'ISO13584_IEC61360_class_constraint_schema.

7.6.2 Unique_constraint_id

La règle **unique_constraint_id** affirme que deux **constraint_identifier** associés à deux **constraint** différentes ont des valeurs différentes.

EXPRESS specification:

```

*)
RULE unique_constraint_id FOR(constraint);
WHERE
    QUERY(c1 <* constraint |
        SIZEOF(QUERY(c2 <* constraint |
            c1.constraint_id = c2.constraint_id))>1) = [];
END_RULE; -- unique_constraint_id
(*)

*)
END_SCHEMA; -- ISO13584_IEC61360_class_constraint_schema

(*)

```

8 ISO13584_IEC61360_item_class_case_of_schema

8.1 Vue d'ensemble

Le présent article définit l'exigence pour l'ISO13584_IEC61360_item_class_case_of_schema. La déclaration EXPRESS suivante introduit le bloc ISO13584_IEC61360_item_class_case_of_schema et identifie les références externes nécessaires.

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_item_class_case_of_schema;

REFERENCE FROM ISO13584_IEC61360_dictionary_schema
    (all_class_descriptions_reachable,
     class,
     class_BSU,
     data_type_BSU,
     item_class,
     property_BSU);

REFERENCE FROM ISO13584_IEC61360_class_constraint_schema
    (constraint,
     integrity_constraint,
     context_restriction_constraint,
     property_constraint,
     domain_constraint);

REFERENCE FROM ISO13584_extended_dictionary_schema
    (document_BSU,
     table_BSU,
     visible_properties,
     applicable_properties,
     visible_types,
     applicable_types,
     data_type_named_type);

( *

```

NOTE Les schémas conceptuels référencés ci-dessus peuvent être consultés dans les documents suivants:

ISO13584_IEC61360_dictionary_schema	CEI 61360-2
(qui est dupliqué pour la commodité dans ce document).	
ISO13584_IEC61360_class_constraint_schema	CEI 61360-2
(qui est dupliqué pour la commodité dans ce document).	
ISO13584_extended_dictionary_schema	ISO 13584-24:2003

8.2 Introduction à l'ISO13584_IEC61360_item_class_case_of_schema

Pour des raisons de modularité, le modèle complet de dictionnaire commun ISO/CEI est divisé en plusieurs documents. Le modèle noyau pour ontologies de produits est défini dans la CEI 61360-2 et dupliqué dans la présente partie de la CEI 61360. Les ressources pour étendre ce modèle, y compris la représentation d'instance, la représentation de document, le modèle fonctionnel, les vues fonctionnelles et la représentation des tables, sont définies dans l'ISO 13584-24:2003. Les divers niveaux normalisés de mise en œuvre du modèle complet ISO/CEI, appelés "classes de conformité", sont définis dans l'ISO 13584-25, et dupliqués à des fins informatives dans la CEI 61360-5. Le premier niveau correspond précisément au contenu de la présente partie de la CEI 61360 plus la ressource pour des valeurs structurées en agrégat définies dans l'ISO 13584-25. D'autres classes de conformité incluent de plus en plus de ressources issues de l'ISO 13584-24:2003.

Définir une classe d'articles comme étant une case-of d'une autre classe d'articles est de plus en plus utilisé par des applications basées sur le modèle de dictionnaire commun ISO13584/CEI61360. En outre, la présente édition de la présente partie de la CEI 61360 a

nécessité un changement du modèle d'information de ce concept. Par conséquent, il a été décidé de déplacer l'entité EXPRESS correspondante, appelée **item_class_case_of**, et sa superclasse, appelée **a_priori_semantic_relationship**, de l'ISO 13584-24:2003 vers la présente partie de la CEI 61360. Ces entités sont incluses dans un nouveau schéma, appelé **ISO13584_IEC61360_item_class_case_of_schema**. Les normes ISO 13584-24:2003 et ISO 13584-25 seront mises à jour en conséquence au moyen d'un corrigendum technique.

8.3 Définitions d'entités de l'ISO13584_IEC61360_item_class_case_of_schema

8.3.1 Relation sémantique *a priori*

Une **a_priori_semantic_relationship** est une **class** abstraite qui est définie sur la base d'autres classes et qui peut importer des propriétés, des types de données, des tableaux et des documents contenus dans ces classes. Elle importe aussi toutes les **contraintes** qui limitaient le domaine des propriétés importées dans les classes à partir desquelles elles sont importées. Cette ressource abstraite est destinée à être sous-typée par des classes. Lorsqu'une classe spécialise une **a_priori_semantic_relationship**, les propriétés, types de données, tableaux ou documents dont les définitions sont importées par héritage d'une **a_priori_semantic_relationship** deviennent applicables à la classe qui les importe. En particulier, les propriétés et les types de données qui sont ainsi importés sont autorisés à être utilisés pour décrire des instances de classes. De plus, le fait, pour un produit, d'avoir un aspect qui corresponde à chaque propriété importée est un critère nécessaire pour être membre de la classe.

NOTE 1 Toutes les propriétés et tous les types de données importés deviennent directement applicables sans être visibles. Ainsi, ils ne sont pas retournés par la fonction **compute_known_visible_properties** ou **compute_known_visible_data_types**.

NOTE 2 La relation d'héritage est un exemple notoire de relation sémantique entre des classes modélisées selon le paradigme orienté objet. Toutes les propriétés et autres ressources définies dans une classe s'appliquent habituellement de façon implicite à toutes ses sous-classes. Cette relation est utilisée dans la série ISO 13584 lorsque toutes les propriétés, tous les types de données, tous les tableaux ou tous les documents visibles (respectivement applicables) à certaines classes sont implicitement visibles (respectivement applicables) à toutes ses sous-classes. Comme d'habitude, dans l'ISO 13584, cet héritage est implicite (c'est-à-dire: non déclaré au moyen d'une **a_priori_semantic_relationship**) et global (c'est-à-dire: toutes les propriétés et tous les types de données sont hérités par toutes ses sous-classes). Une **a_priori_semantic_relationship** permet de définir d'autres relations sémantiques qui sont utiles dans le domaine d'application de l'ISO 13584, et en particulier, la relation case-of qui permet une importation explicite et partielle de propriétés, et d'autres ressources définies dans une classe.

EXPRESS specification:

```
*)
ENTITY a_priori_semantic_relationship
ABSTRACT SUPERTYPE
SUBTYPE OF(class);
    referenced_classes: SET [1:?] OF class_BSU;
    referenced_properties: LIST [0:?] OF property_BSU;
    referenced_data_types: SET [0:?] OF data_type_BSU;
    referenced_tables: SET [0:?] OF table_BSU;
    referenced_documents: SET [0:?] OF document_BSU;
    referenced_constraints: SET [0:?] OF
constraint_or_constraint_id;
WHERE
    WR1: QUERY (cons <* SELF.referenced_constraints
        | NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.ISO_29002_IRDI_type') IN TYPEOF(cons))
        AND NOT (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            + '.PROPERTY_CONSTRAINT') IN TYPEOF(cons)))
        = [];
    WR2: QUERY (cons <* SELF.referenced_constraints
```

```

      | (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
      +'.PROPERTY_CONSTRAINT') IN TYPEOF (cons))
      AND NOT (cons\property_constraint.constrained_property
      IN SELF.referenced_properties))
      = [];
WR3: compute_known_referenced_property_constraints(SELF)
      <= SELF.referenced_constraints;
WR4: QUERY(prop <* SELF.referenced_properties
      | QUERY(cl <* SELF.referenced_classes
      | visible_properties(cl, [prop])
      OR applicable_properties(cl, [prop]))
      = []) = [];
WR5: QUERY(typ <* SELF.referenced_data_types
      | QUERY(cl <* SELF.referenced_classes
      | visible_types(cl, [typ])
      OR applicable_types(cl, [typ]))
      = []) = [];
END_ENTITY; -- a_priori_semantic_relationship
(*

```

Définitions des attributs:

referenced_classes: la/les classe(s) à partir de laquelle/desquelles les propriétés, types de données, tableaux ou documents sont importés.

NOTE 3 La classe à partir de laquelle des propriétés, types de données, tableaux ou documents sont importés ne peut pas être déduite de l'identification des propriétés, types de données, tableaux ou documents importés, car ils peuvent être importés d'une classe où ils sont hérités ou importés. Par exemple, dans la CEI 61360-DB, "input-voltage" (tension d'entrée) est une propriété visible au niveau de la racine de la classification CEI. Si une classe fournisseur importe la propriété "input-voltage" d'une classe "transistor" CEI, cela signifie que (1) la classe fournisseur définit un transistor, et (2) ces transistors sont décrits au moyen de la propriété "input voltage".

referenced_properties: les propriétés dont les définitions sont importées au moyen de l'entité **a_priori_semantic_relationship**.

NOTE 4 L'ordre de la liste définit l'ordre par défaut pour afficher des propriétés importées au cours de l'accès utilisateur à divers sous-types de **a_priori_semantic_relationship**.

referenced_data_types: les types de données dont les définitions sont importées par le biais de l'entité **a_priori_semantic_relationship**.

referenced_tables: les tableaux dont les définitions sont importées par le biais de l'entité **a_priori_semantic_relationship**.

NOTE 5 Les ressources et les règles détaillées sur l'utilisation de tableaux sont définies dans l'ISO 13584-24:2003. Elles ne sont utilisées ni dans la présente partie de la CEI 61360 ni dans les modèles intégrés documentés dans l'ISO 13584-32 (OntoML) et l'ISO 13584-25.

referenced_documents: les documents dont les définitions sont importées par le biais de l'entité **a_priori_semantic_relationship**.

NOTE 6 Les ressources et les règles détaillées sur l'utilisation des documents sont définies dans l'ISO 13584-24:2003. Elles sont utilisées dans les modèles intégrés documentés dans l'ISO 13584-32 (OntoML) et l'ISO 13584-25.

referenced_constraints: les **property_constraint** qui s'appliquent aux diverses propriétés importées.

NOTE 7 Contrairement aux entités référencées, les contraintes **referenced_constraints** ne peuvent pas être sélectionnées lorsque **a_priori_semantic_relationship** est conçue. Ces contraintes sont toutes les contraintes qui

affectent un nombre quelconque de propriétés définies dans l'attribut **referenced_properties** dans n'importe quelle classe de l'attribut **referenced_classes**.

Propositions formelles:

WR1: toutes les **referenced_constraints** qui ne sont pas IRDI doivent être des **property_constraint**.

WR2: toutes les **referenced_constraints** doivent contraindre les propriétés qui sont importées par le biais de l'attribut **referenced_properties**.

WR3: toutes les **property_constraint** qui contraignent l'une des propriétés **referenced_properties** dans n'importe laquelle des classes **referenced_classes** doivent être importées par le biais de l'attribut **referenced_constraints**.

WR4: les propriétés importées définies par l'attribut **referenced_properties** doivent être visibles ou applicables pour l'une des classes appartenant à l'attribut **referenced_classes**.

WR5: les types importés définis par l'attribut **referenced_data_types** doivent être visibles ou applicables pour l'une des classes appartenant à l'attribut **referenced_classes**.

Propositions informelles:

IP1: toutes les **constraint** qui sont représentées par des **constraint_identifier** dans l'ensemble de **referenced_constraints** doivent correspondre aux **property_constraint** qui contiennent l'une des propriétés **referenced_properties** dans l'une des classes **referenced_classes**. Cette contrainte ne doit pas être représentée, dans le même ensemble **referenced_constraints**, tant comme **property_constraint** que comme **constraint_identifier**.

NOTE 8 Une contrainte représentée comme une **property_constraint** dans l'une des classes **referenced_classes** peut être représentée dans l'ensemble **referenced_constraints** soit comme une **property_constraint**, soit comme une **constraint_identifier**.

IP2: toutes les contraintes qui sont représentées par un **constraint_identifier** dans l'une des classes **referenced_classes**, mais dont la **constraint** correspondante est une **property_constraint** qui contraint l'une des propriétés importées par le biais de l'attribut **referenced_properties**, doivent être représentées par leur **constraint_identifier** dans l'ensemble **referenced_constraints**.

NOTE 9 Ces deux règles informelles assurent que l'ensemble **referenced_constraints** de contraintes est l'union des ensembles de **property_constraint** définis dans les diverses classes **referenced_classes** dont la **constrained_property** appartient à l'ensemble **referenced_properties**, même lorsque le contexte d'échange ne contient pas les définitions de toutes les classes impliquées dans **a_priori_semantic_relationship** et lorsque certaines **constraint** sont seulement représentées par leurs **constraint_identifier**.

8.3.2 **Item_class_case_of**

Une **item_class_case_of** est la description d'une classe d'articles qui est définie comme étant une "is-case-of" de quelque autre(s) classe(s) d'articles.

NOTE 1 Une entité **item_class_case_of** définit une relation sémantique *a priori*.

EXPRESS specification:

```
* )
ENTITY item_class_case_of
SUBTYPE OF (item_class, a_priori_semantic_relationship);
    is_case_of: SET [1:?] OF class_BSU;
    imported_properties: LIST [0:?] OF property_BSU;
```

```

imported_types: SET [0:?] OF data_type_BSU;
imported_tables: SET [0:?] OF table_BSU;
imported_documents: SET [0:?] OF document_BSU;
imported_constraints: SET [0:?] OF
constraint_or_constraint_id;
DERIVE
  SELF\a_priori_semantic_relationship.referenced_classes:
    SET [1:?] OF class_BSU := SELF.is_case_of;
  SELF\a_priori_semantic_relationship.referenced_properties:
    LIST [0:?] OF property_BSU := SELF.imported_properties;
  SELF\a_priori_semantic_relationship.referenced_data_types:
    SET [0:?] OF data_type_BSU := SELF.imported_types;
  SELF\a_priori_semantic_relationship.referenced_tables:
    SET [0:?] OF table_BSU := SELF.imported_tables;
  SELF\a_priori_semantic_relationship.referenced_documents:
    SET [0:?] OF document_BSU := SELF.imported_documents;
  SELF\a_priori_semantic_relationship.referenced_constraints:
    SET [0:?] OF property_constraint
    := SELF.imported_constraints;
WHERE
  WR1: superclass_of_item_is_item(SELF);
  WR2: check_is_case_of_referenced_classes_definition(SELF);
  WR3: QUERY(p <* SELF\class.sub_class_properties
    | NOT((p IN SELF.described_by)
    OR (p IN SELF.imported_properties))) = [];
  WR4: QUERY(p <* SELF\class.sub_class_properties
    | (p IN SELF.imported_properties)
    AND (QUERY(cl<*SELF.is_case_of
    | all_class_descriptions_reachable(cl) AND
    (p IN compute_known_applicable_properties(cl)) AND
    (NOT is_class_valued_property(p, cl))<>[]))
    =[]);
  WR5: QUERY(ccv <* SELF\class.class_constant_values
    | (ccv.super_class_defined_property
    IN SELF.imported_properties)
    AND (QUERY(cl<*SELF.is_case_of
    | all_class_descriptions_reachable(cl) AND
    (ccv.super_class_defined_property
    IN compute_known_applicable_properties(cl)) AND
    (QUERY (v<*class_value_assigned(
    ccv.super_class_defined_property, cl)
    |v<> ccv.assigned_value) <> []))<>[]))
    =[]);
  WR6: QUERY(prop <* imported_properties
    | (QUERY(cl<*SELF.is_case_of
    | is_class_valued_property(prop, cl)) <>[]))
    AND NOT is_class_valued_property(prop,
SELF.identified_by))
    =[];
  WR7: QUERY(ccv <* SELF\class.class_constant_values
    | QUERY(cl<*SELF.is_case_of
    | (class_value_assigned
    (ccv.super_class_defined_property, cl) <> []))
    AND (QUERY(v <* class_value_assigned

```



```

(ccv.super_class_defined_property, cl)
| v <> ccv.assigned_value)<>[])) <> [])
= [];
END_ENTITY; -- item_class_case_of
(*

```

Définitions des attributs:

is_case_of: la/les **item_class** dont la présente **item_class** est "is-case-of".

imported_properties: la liste des propriétés qui sont importées de(s) **item_class** dont l'**item_class** définie est "is-case-of".

imported_types: l'ensemble des types de données qui sont importés de(s) **item_class** dont l'**item_class** définie est "is-case-of".

imported_tables: l'ensemble des **table_BSU** qui sont importés de(s) **item_class** dont l'**item_class** définie est "is-case-of".

imported_documents: l'ensemble des **document_BSU** qui sont importés de(s) **item_class** dont l'**item_class** définie est "is-case-of".

imported_constraints: l'ensemble de **property_constraint** ou de **constraint_id** qui sont importés de(s) **item_class** dont l'**item_class** définie est "is-case-of".

NOTE 2 Contrairement à d'autres entités importées, les contraintes **imported_constraints** ne peuvent pas être sélectionnées lorsque **item_class_case_of** est conçue. Ces contraintes sont toutes les contraintes qui limitent les domaines d'un nombre quelconque de propriétés définies dans l'**imported_properties** dans les classes de l'attribut **is_case_of** à partir desquelles elles sont importées. Ceci est spécifié dans une règle WHERE d'une **a_priori_semantic_relationship**.

Propositions formelles:

WR1: la superclasse de **item_class_case_of** doit être une **item_class**.

WR2: une **item_class_case_of** doit être une/des **item_class** "case-of".

WR3: **sub_class_properties** doit appartenir soit à la liste **described_by**, soit à la liste **imported_properties**.

WR4: toutes les propriétés de valeurs d'une classe déclarées au moyen de **sub_class_properties** qui sont des **imported_properties** doivent être des propriétés de valeur d'une classe dans toutes les classes **is-case-of** où elles sont applicables.

WR5: les valeurs assignées à une propriété importée au moyen de l'affectation de **class_constant_value** ne doivent pas être différentes de la valeur possible assignée à la même propriété dans les classes référencées.

WR6: toutes les **imported_properties** qui sont des propriétés de valeurs d'une classe dans une classe issue de l'ensemble **is_case_of** de classes, doivent être de propriétés de valeur d'une classe dans la classe courante.

WR7: toutes les **imported_properties** qui ont une valeur **class_constant_value** qui leur est assignée dans une classe issue de l'ensemble **is_case_of** de classes, doivent avoir la même **class_constant_value** assignée dans la classe courante.

8.4 Définitions de fonctions de l'ISO13584_IEC61360_item_class_case_of_schema

8.4.1 Généralités

Le présent paragraphe contient des fonctions qui sont référencées dans les articles WHERE pour affirmer la cohérence des données ou qui fournissent des ressources pour le développement d'applications.

8.4.2 Fonction compute_known_property_constraints

La fonction **compute_known_property_constraints** calcule l'ensemble des **property_constraint** qui s'applique aux propriétés d'un ensemble de classes. Les contraintes représentées par leurs identificateurs ne sont pas retournées. Lorsque la définition de certaines classes n'est pas disponible, la fonction retourne seulement les entités **property_constraint** qui peuvent être calculées.

NOTE Lorsque l'attribut **dictionary_definition** d'une classe n'est pas disponible dans le même contexte d'échange (un contexte d'échange PLIB n'est jamais supposé être complet), sa propre superclasse peut ne pas être connue. Par conséquent, les contraintes définies par cette superclasse ne peuvent pas être calculées par la fonction **compute_known_property_constraints**. Au contraire, lorsque toutes les superclasses "is-a" d'une classe sont disponibles dans le même contexte d'échange, toutes les contraintes qui s'appliquent à cette classe peuvent être calculées par une seule analyse traverse de son arbre d'héritage "is-a", même lorsque certaines de ces superclasses importent des propriétés au moyen de **a_priori_semantic_relationship** telles que **item_class_case_of**.

EXPRESS specification:

```

*)
FUNCTION compute_known_property_constraints(classes: SET OF
class_BSU):
    SET OF property_constraint;

LOCAL
    s: SET OF property_constraint := [];
END_LOCAL;

REPEAT nb := 1 TO SIZEOF (classes);
    IF SIZEOF(classes[nb].definition)=1
    THEN
        REPEAT i := 1 TO
            SIZEOF(classes[nb].definition[1]\class.constraints);
            IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
                +'.PROPERTY_CONSTRAINT')
                IN TYPEOF
                    (classes[nb].definition[1]\class.constraints[i]))
                THEN
                    s := s +
classes[nb].definition[1]\class.constraints[i];
                END_IF;
            END_REPEAT;

        IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
            + 'A_PRIORI_SEMANTIC_RELATIONSHIP')
            IN TYPEOF (classes[nb].definition[1]))
        THEN
            REPEAT i := 1 TO
                SIZEOF(classes[nb].definition[1]

```

```

\apriori_semantic_relationship
.referenced_constraints);

    IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
        +'.PROPERTY_CONSTRAINT') IN TYPEOF
        (classes[nb].definition[1]
        \apriori_semantic_relationship
        .referenced_constraints[i]))
    THEN
        s := s + classes[nb].definition[1]
        \apriori_semantic_relationship
        .referenced_constraints [i];
    END_IF;
END_REPEAT;
END_IF;

IF EXISTS(classes[nb].definition[1]\class.its_superclass)
THEN
    s := s + compute_known_property_constraints(

[classes[nb].definition[1]\class.its_superclass]);
END_IF;

END_IF;
END_REPEAT;
RETURN(s);

END_FUNCTION; -- compute_known_property_constraints
(*)

```

8.4.3 Fonction compute_known_referenced_property_constraints

La fonction **compute_known_referenced_property_constraints** calcule toutes les entités **property_constraints** qu'il convient qu' **ap** **a_priori_semantic_relationship** ait importées en calculant toutes les contraintes qui s'appliquent à une propriété qui est importée par le biais de l'attribut **referenced_properties** de **ap**, et qui sont définies ou héritées dans n'importe quelle classe de **referenced_classes** de **ap** dont **dictionary_definition** de **classe** est disponible dans le même contexte d'échange.

NOTE 1 Dans **a_priori_semantic_relationship**, il convient que toutes les **property_constraint** définies ou héritées dans les classes référencées par leur attribut **referenced_classes** qui s'appliquent à une propriété qui est importée par le biais de l'attribut **referenced_properties** de **a_priori_semantic_relationship** soient importées par le biais de l'attribut **referenced_constraints**.

NOTE 2 Lorsque l'attribut **dictionary_definition** d'une classe appartenant à l'attribut **referenced_classes** de **ap** n'est pas disponible dans le même contexte d'échange que **ap** (un contexte d'échange PLIB n'est jamais supposé être complet), les contraintes appartenant à cette classe ne peuvent pas être calculées. Ainsi, le résultat de la fonction **compute_known_referenced_property_constraints** peut seulement être un sous-ensemble des contraintes qu'il convient d'importer par **ap**.

NOTE 3 Lorsque les attributs **dictionary_definition** de toutes les classes **referenced_classes** de **ap** sont disponibles dans le même contexte d'échange, et lorsque aucune contrainte n'est représentée par un seul **constraint_identifieur**, la fonction **compute_known_referenced_property_constraints** retourne exactement toutes les contraintes qu'il convient d'importer par **ap**.

EXPRESS specification:

```

*)
FUNCTION compute_known_referenced_property_constraints(
    ap: a_priori_semantic_relationship):
    SET OF property_constraint;

LOCAL
    s: SET OF property_constraint := []; -- result
    prop: SET OF property_BSU :=
        list_to_set(ap.referenced_properties); --imported
    properties
    cl: SET OF class_BSU :=
        ap.referenced_classes; --source of importation
    cons: SET OF property_constraint
        := compute_known_property_constraints(cl);
        -- toutes les property_constraints existant dans les
diverses
        -- classes issues de cl qui peuvent être calculées dans
le présent
        -- contexte d'échange.
END_LOCAL;

    REPEAT n_cons := 1 TO SIZEOF(cons);
        IF cons[n_cons].constrained_property IN prop
        THEN
            s := s + cons[n_cons];
        END_IF;
    END_REPEAT;
RETURN(s);

END_FUNCTION; -- compute_known_referenced_property_constraints
(*)

```

8.4.4 Fonction superclass_of_item_is_item

La fonction **superclass_of_item_is_item** s'assure que la superclasse d'une **item_class** **cl**, si elle existe, est une **item_class**.

Si la **classe** associée à **class_BSU** ne peut pas être calculée, la fonction retourne UNKNOWN.

EXPRESS specification:

```

*)
FUNCTION superclass_of_item_is_item(cl: item_class): LOGICAL;

IF NOT EXISTS(cl\class.its_superclass)
THEN
    RETURN(TRUE);
END_IF;

IF SIZEOF(cl\class.its_superclass.definition) = 0
THEN
    RETURN(UNKNOWN);
END_IF;

RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.ITEM_CLASS')
    IN TYPEOF(cl\class.its_superclass.definition[1]));

```

```
END_FUNCTION; -- superclass_of_item_is_item
(*
```

8.4.5 Fonction `check_is_case_of_referenced_classes_definition`

La fonction **`check_is_case_of_referenced_classes_definition`** retourne TRUE si l'ensemble **`item_class_case_of_is_case_of`** de définition(s) du dictionnaire des classes référencées est de type compatible avec l'instance donnée de **`item_class_case_of`** de **`cl`**. Autrement, elle retourne FALSE.

EXPRESS specification:

```
*)
FUNCTION check_is_case_of_referenced_classes_definition(
    cl: item_class_case_of): BOOLEAN;
LOCAL
    class_def_ok: BOOLEAN := TRUE;
END_LOCAL;

REPEAT i := 1 TO SIZEOF(cl.is_case_of);
    IF (SIZEOF(cl.is_case_of[i].definition) = 1)
    THEN
        IF (NOT('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.ITEM_CLASS'
            IN TYPEOF(cl.is_case_of[i].definition[1])))
        THEN
            class_def_ok := FALSE;
        END_IF;
    END_IF;
END_REPEAT;

RETURN(class_def_ok);

END_FUNCTION; -- check_is_case_of_referenced_classes_definition
(*
```

8.5 Définitions de règle de l'ISO13584_IEC61360_item_class_case_of_schema

8.5.1 Généralités

Le présent paragraphe définit la règle dans l'ISO13584_IEC61360_item_class_case_of_schema.

8.5.2 Règle `imported_properties_are_visible_or_applicable_rule`

La règle **`imported_properties_are_visible_or_applicable_rule`** s'assure que lorsqu'une propriété est importée par une classe au moyen de **`a_priori_semantic_relationship`**, cette propriété est visible ou applicable pour la classe à partir de laquelle elle est importée.

NOTE Les propriétés applicables comprennent les propriétés importées par le biais d'une relation sémantique. Cette règle permet d'importer des propriétés issues d'une classe dans laquelle elles étaient déjà importées.

EXPRESS specification:

```
*)
RULE imported_properties_are_visible_or_applicable_rule FOR(
```

```

a_priori_semantic_relationship, property_DET);
WHERE
  WR1: QUERY(rel <* a_priori_semantic_relationship
    | QUERY(prop <* rel.referenced_properties
    | QUERY(cl <* rel.referenced_classes
    | NOT visible_properties(cl, [prop])
    AND NOT applicable_properties(cl, [prop]))
    = rel.referenced_classes) = [])
    = a_priori_semantic_relationship;
END_RULE; -- imported_properties_are_visible_or_applicable_rule
(*)

```

8.5.3 Règle imported_data_types_are_visible_or_applicable_rule

La règle **imported_data_types_are_visible_or_applicable_rule** s'assure que lorsqu'un type de données est importé par une classe au moyen d'**a_priori_semantic_relationship**, ce type de données est visible ou applicable pour la classe à partir de laquelle elle est importée.

NOTE Les types de données applicables comprennent les types de données importés par le biais d'une relation sémantique. Cette règle permet d'importer des types de données issus d'une classe dans laquelle ils étaient déjà importés.

EXPRESS specification:

```

*)
RULE imported_data_types_are_visible_or_applicable_rule FOR(
  a_priori_semantic_relationship, data_type_element);
WHERE
  WR1: QUERY(rel <* a_priori_semantic_relationship
    | QUERY(typ <* rel.referenced_data_types
    | QUERY(cl <* rel.referenced_classes
    | NOT visible_types(cl, [typ])
    AND NOT applicable_types(cl, [typ]))
    = rel.referenced_classes) = [])
    = a_priori_semantic_relationship;
END_RULE; -- imported_data_types_are_visible_or_applicable_rule
(*)

```

8.5.4 Règle allowed_named_type_usage_rule

La règle **allowed_named_type_usage_rule** est relative à l'utilisation d'un type nommé. Elle énonce que seuls des types qui sont applicables à une classe peuvent être utilisés pour spécifier le domaine des propriétés déclarées par une classe, par le biais de son attribut **described_by**.

EXPRESS specification:

```

*)
RULE allowed_named_type_usage_rule FOR(class);
LOCAL
  named_type_usage_allowed: LOGICAL := TRUE;
  is_app: LOGICAL;
  prop: property_bsu;
  cl: class;
  dtnt: SET[0:1] OF data_type_bsu := [];
END_LOCAL;

```

```

REPEAT i := 1 TO SIZEOF(class);
  cl := class[i];
  REPEAT j := 1 TO SIZEOF(class[i].described_by);
    prop := cl.described_by[j];
    dtnt := data_type_named_type(prop);

    IF (SIZEOF(dtnt) = 1) THEN
      is_app := applicable_types(cl.identified_by, dtnt);
      IF (NOT is_app) THEN
        named_type_usage_allowed := FALSE;
      END_IF;
    END_IF;
  END_REPEAT;
END_REPEAT;

WHERE
  WR1: named_type_usage_allowed;
END_RULE; -- allowed_named_type_usage_rule
(*)

*)
END_SCHEMA; -- ISO13584_IEC61360_item_class_case_of_schema
(*)

```

Annexe A (informative)

Exemple de fichier physique

A.1 Objectif

La présente annexe donne un certain nombre de fragments d'un fichier physique pour échanger les données de la CEI 61360-DB. Il vise à montrer l'utilisation du modèle EXPRESS à l'Article 5 L"ISO13584_IEC61360_dictionary_schema" et l'ISO 10303-21 ensemble pour échanger des données correspondantes.

A.2 En-tête de fichier

```
*/
ISO-10303-21;
HEADER;
FILE_DESCRIPTION(('Exemple de fichier physique'), '2;1');
FILE_NAME('example.spf', '2007-07-18', ('IEC SC3D WG2'), (),
'Version 1', '', '');
FILE_SCHEMA(('example_schema'));
ENDSEC;
DATA;
/*
```

A.3 Données fournisseur

```
*/
#1=SUPPLIER_BSU('112/2///61360_4_1', *); /*conformément à l'ISO 13584-
26*/
#2=SUPPLIER_ELEMENT(#1, #3, '01', $, $, $, #4, #5);
#3=DATES('1994-09-16', '1994-09-16', $);
#4=ORGANIZATION('CEI', 'Agence de maintenance CEI', 'L'Agence de
maintenance de la CEI');
#5=ADDRESS('à déterminer', $, $, $, $, $, $, $, $, $, $, $);
#10=SUPPLIER_BSU('112/3///_00', *); /* ICS ISO/CEI */
/*
```

A.4 Données de la classe-racine

La classe-racine AAA000 de la CEI fournit une portée des noms correspondants à la CEI 61360-DB. Elle couvre deux arbres, un pour les matériaux, un pour les composants. Elle est définie comme une **item_class**.

```
*/
#90=CLASS_BSU('00', '001', #10);
#100=CLASS_BSU('AAA000', '001', #1);
#101=ITEM_CLASS(#100, #3, '01', $, $, $, #102, TEXT('La classe-racine
CEI qui donne une portée de noms correspondant à la CEI 61360-DB. Elle
couvre deux arbres, un pour les matériaux, un pour les composants'),
$, $, $, #90, (#110), (), (), $, (), (#110), (), $, $, $);
```



```
#102=ITEM_NAMES(LABEL('Classe-racine CEI'), (), LABEL('Racine CEI'),
$, $);
#110=PROPERTY_BSU('AAE000', '001', #100);
#111=NON_DEPENDENT_P_DET(#110, #3, '01', $, $, $, #112, TEXT('le type
d'arbre: matériau ou composant'), $, $, $, $, (), $, $, #113, $);
#112=ITEM_NAMES(LABEL('type d'arbre'), (), LABEL('tree type'), $, $);
#113=NON_QUANTITATIVE_CODE_TYPE((), 'A..8', #114);
#114=VALUE_DOMAIN((#120,#122), $, $, (), $, $);
#120=DIC_VALUE(VALUE_CODE_TYPE('MATÉRIAU'), #121, $, $, $, $, $, $);
#121=ITEM_NAMES(LABEL('arbre de matériaux'), (), LABEL('arbre de
mat'), $, $);
#122=DIC_VALUE(VALUE_CODE_TYPE('COMPOS'), #123, $, $, $, $, $, $);
#123=ITEM_NAMES(LABEL('arbre de composant'), (), LABEL('arbre de
comp'), $, $);
/*
```

A.5 Données des matériaux

```
*/
#200=CLASS_BSU('AAA218', '001', #1);
#201=ITEM_CLASS(#200, #3, '01', $, $, $, #202, TEXT('classe-racine de
l'arbre de matériaux'), $, $, $, #100, (#210,#230), (), (), $, (),
(#210), (#205), $, 'MATERIAL', $);
#202=ITEM_NAMES(LABEL('classe-racine de matériaux'), (), LABEL('racine
de matériaux'), $, $);
#205=CLASS_VALUE_ASSIGNMENT(#110, STRING_VALUE('MATERIAL'));
#210=PROPERTY_BSU('AAF311', '005', #100);
#211=NON_DEPENDENT_P_DET(#210, #3, '01', $, $, $, #212, TEXT('code du
type de matériau'), $, $, $, $, (), $, 'A57', #213, $);
#212=ITEM_NAMES(LABEL('type de matériau'), (), LABEL('type de
matériau'), $, $);
#213=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #214);
#214=VALUE_DOMAIN((#220,#222,#224,#226), $, $, (), $, $);
#220=DIC_VALUE(VALUE_CODE_TYPE('ACO'), #221, $, $, $, $, $, $);
#221=ITEM_NAMES(LABEL('acoustique'), (), LABEL('acoustique'), $, $);
#222=DIC_VALUE(VALUE_CODE_TYPE('MG'), #223, $, $, $, $, $, $);
#223=ITEM_NAMES(LABEL('magnétique'), (), LABEL('magnétique'), $, $);
#224=DIC_VALUE(VALUE_CODE_TYPE('OP'), #225, $, $, $, $, $, $);
#225=ITEM_NAMES(LABEL('optique'), (), LABEL('optique'), $, $);
#226=DIC_VALUE(VALUE_CODE_TYPE('TH'), #227, $, $, $, $, $, $);
#227=ITEM_NAMES(LABEL('thermoélectrique'), (), LABEL('th-électrique'),
$, $);
#230=PROPERTY_BSU('AAF286', '005', #100);
#231=NON_DEPENDENT_P_DET(#230, #3, '01', $, $, $, #232, TEXT('La masse
volumique nominale (en kg/m**3) d'un matériau.'), $, $, $, #233, (),
$, 'K02', #234, $);
#232=ITEM_NAMES(LABEL('masse volumique'), (), LABEL('masse
volumique'), $, $);
#233=MATHEMATICAL_STRING('$r_d', '&rho;<sub>d</sub>');
#234=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #235, $, $, $);
```

```
#235=DIC_UNIT(#236, $);
#236=DERIVED_UNIT((#239,#237));
#237=DERIVED_UNIT_ELEMENT(#238, 1.);
#238=SI_UNIT(*, .KILO., .GRAM.);
#239=DERIVED_UNIT_ELEMENT(#240, -3.);
#240=SI_UNIT(*, $, .METRE.);
/*
```

A.6 Données des composants

```
*/
#300=CLASS_BSU('EEE000', '001', #1);
#301=ITEM_CLASS(#300, #3, '01', $, $, $, #302, TEXT('classe-racine de
l'arbre de composants'), $, $, $, #100, (#310,#330,#350), (), (), $,
(), (#310), (#305), $, 'COMPONS', $);
#302=ITEM_NAMES(LABEL('classe-racine de composants'), (),
LABEL('racine de composants'), $, $);
#305=CLASS_VALUE_ASSIGNMENT(#110, STRING_VALUE('COMPONS'));
#310=PROPERTY_BSU('AAE001', '005', #100);
#311=NON_DEPENDENT_P_DET(#310, #3, '01', $, $, $, #312, TEXT('Code de
la principale classe fonctionnelle à laquelle un composant
appartient'), $, $, $, $, (), $, 'A52', #313, $);
#312=ITEM_NAMES(LABEL('principale classe de composant'), (),
LABEL('principale classe'), $, $);
#313=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #314);
#314=VALUE_DOMAIN((#320,#322,#324,#326), $, $, (), $, $);
#320=DIC_VALUE(VALUE_CODE_TYPE('EE'), #321, $, $, $, $, $, $);
#321=ITEM_NAMES(LABEL('EE (électrique/ électronique)'), (),
LABEL('EE'), $, $);
#322=DIC_VALUE(VALUE_CODE_TYPE('EM'), #323, $, $, $, $, $, $);
#323=ITEM_NAMES(LABEL('électromécanique'), (), LABEL('électroméc'), $,
$);
#324=DIC_VALUE(VALUE_CODE_TYPE('ME'), #325, $, $, $, $, $, $);
#325=ITEM_NAMES(LABEL('mécanique'), (), LABEL('mécanique'), $, $);
#326=DIC_VALUE(VALUE_CODE_TYPE('MP'), #327, $, $, $, $, $, $);
#327=ITEM_NAMES(LABEL('pièce magnétique'), (), LABEL('magnétique'), $,
$);
#330=PROPERTY_BSU('AAF267', '005', #100);
#331=NON_DEPENDENT_P_DET(#330, #3, '01', $, $, $, #332, TEXT('La
distance nominale (en m) entre l'intérieur de deux bandes utilisées
pour les produits bandés avec conducteurs axiaux'), $, $, $, #333, (),
$, 'T03', #334, $);
#332=ITEM_NAMES(LABEL('espacement de bande interne'), (),
LABEL('espacement de bande interne'), $, $);
#333=MATHEMATICAL_STRING('b_tape', 'b<sub>tape</sub>');
#334=LEVEL_TYPE((), (.NOM.), #335);
#335=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #336, $, $, $);
#336=DIC_UNIT(#337, $);
#337=SI_UNIT(*, $, .METRE.);
#350=PROPERTY_BSU('AAE022', '005', #100);
```

```
#351=NON_DEPENDENT_P_DET(#350, #3, '01', $, $, $, #352, TEXT('La valeur
telle que spécifiée par le niveau (miNoMax) du diamètre extérieur (en
m) d'un composant avec un corps de section circulaire'), $, $, $,
#353, (), $, 'T03', #354, $);
#352=ITEM_NAMES(LABEL('diamètre extérieur'), (), LABEL('diamètre
ext'), $, $);
#353=MATHEMATICAL_STRING('d_out', 'd<sub>out</sub>');
#354=LEVEL_TYPE((), (.MIN., .NOM., .MAX.), #355);
#355=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #356, $, $, $);
#356=DIC_UNIT(#357, #358);
#357=SI_UNIT(*, $, .METRE.);
#358=MATHEMATICAL_STRING('m', 'm');
/*
```

A.7 Données des composants électriques/électroniques

```
*/
#400=CLASS_BSU('EEE001', '001', #1);
#401=ITEM_CLASS(#400, #3, '01', $, $, $, #402, TEXT('composants
électriques / électroniques'), $, $, $, #300, (#410, #470), (), $,
(), (#410), (#405), $, 'EE', $);
#402=ITEM_NAMES(LABEL('Composants EE'), (), LABEL('Composants EE'), $,
$);
#405=CLASS_VALUE_ASSIGNMENT(#310, STRING_VALUE('EE'));
#410=PROPERTY_BSU('AAE002', '005', #100);
#411=NON_DEPENDENT_P_DET(#410, #3, '01', $, $, $, #412, TEXT('Code de
la catégorie à laquelle appartient le composant
électrique/électronique.'), $, $, $, $, (), $, 'A52', #413, $);
#412=ITEM_NAMES(LABEL('composant de catégorie EE'), (), LABEL('comp
catég EE'), $, $);
#413=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #414);
#414=VALUE_DOMAIN((#420, #422, #424, #426, #428
, #430, #432, #434, #436, #438
, #440), $, $, (), $, $);
#420=DIC_VALUE(VALUE_CODE_TYPE('AMP'), #421, $, $, $, $, $, $);
#421=ITEM_NAMES(LABEL('amplificateur'), (), LABEL('amplificateur'), $,
$);
#422=DIC_VALUE(VALUE_CODE_TYPE('ANT'), #423, $, $, $, $, $, $);
#423=ITEM_NAMES(LABEL('antenne (aerial)'), (), LABEL('antenne (aer)'),
$, $);
#424=DIC_VALUE(VALUE_CODE_TYPE('BAT'), #425, $, $, $, $, $, $);
#425=ITEM_NAMES(LABEL('batterie'), (), LABEL('batterie'), $, $);
#426=DIC_VALUE(VALUE_CODE_TYPE('CAP'), #427, $, $, $, $, $, $);
#427=ITEM_NAMES(LABEL('condensateur'), (), LABEL('condensateur'), $,
$);
#428=DIC_VALUE(VALUE_CODE_TYPE('CND'), #429, $, $, $, $, $, $);
#429=ITEM_NAMES(LABEL('conducteur'), (), LABEL('conducteur'), $, $);
#430=DIC_VALUE(VALUE_CODE_TYPE('DEL'), #431, $, $, $, $, $, $);
#431=ITEM_NAMES(LABEL('ligne de retard'), (), LABEL('ligne de
retard'), $, $);
```

```
#432=DIC_VALUE(VALUE_CODE_TYPE('DID'), #433, $, $, $, $, $, $);
#433=ITEM_NAMES(LABEL('dispositif diode'), (), LABEL('dispositif
diode'), $, $);
#434=DIC_VALUE(VALUE_CODE_TYPE('FIL'), #435, $, $, $, $, $, $);
#435=ITEM_NAMES(LABEL('filtre'), (), LABEL('filtre'), $, $);
#436=DIC_VALUE(VALUE_CODE_TYPE('IC'), #437, $, $, $, $, $, $);
#437=ITEM_NAMES(LABEL('circuit intégré'), (), LABEL('IC'), $, $);
#438=DIC_VALUE(VALUE_CODE_TYPE('IND'), #439, $, $, $, $, $, $);
#439=ITEM_NAMES(LABEL('inducteur'), (), LABEL('inducteur'), $, $);
#440=DIC_VALUE(VALUE_CODE_TYPE('LAM'), #441, $, $, $, $, $, $);
#441=ITEM_NAMES(LABEL('lampe'), (), LABEL('lampe'), $, $);
#470=PROPERTY_BSU('AAE754', '005', #100);
#471=NON_DEPENDENT_P_DET(#470, #3, '01', $, $, $, #472, TEXT('Le nombre
de bornes électriques d'un composant électrique/électronique ou
électromécanique'), $, $, $, #473, (), $, 'Q56', #474, $);
#472=ITEM_NAMES(LABEL('nombre de bornes'), (LABEL('nombre de
broches')), LABEL('nr de bornes'), $, $);
#473=MATHEMATICAL_STRING('N_term', 'N<sub>term</sub>');
#474=INT_TYPE((), 'NR1..4');

ENDSEC;
END-ISO-10303-21;
```

Annexe B
(informative)

Diagramme EXPRESS-G

La présente annexe contient les diagrammes EXPRESS-G pour les Articles 6 à 8.
EXPRESS-G est défini dans l'Annexe A de l'ISO 10303-11:2004.

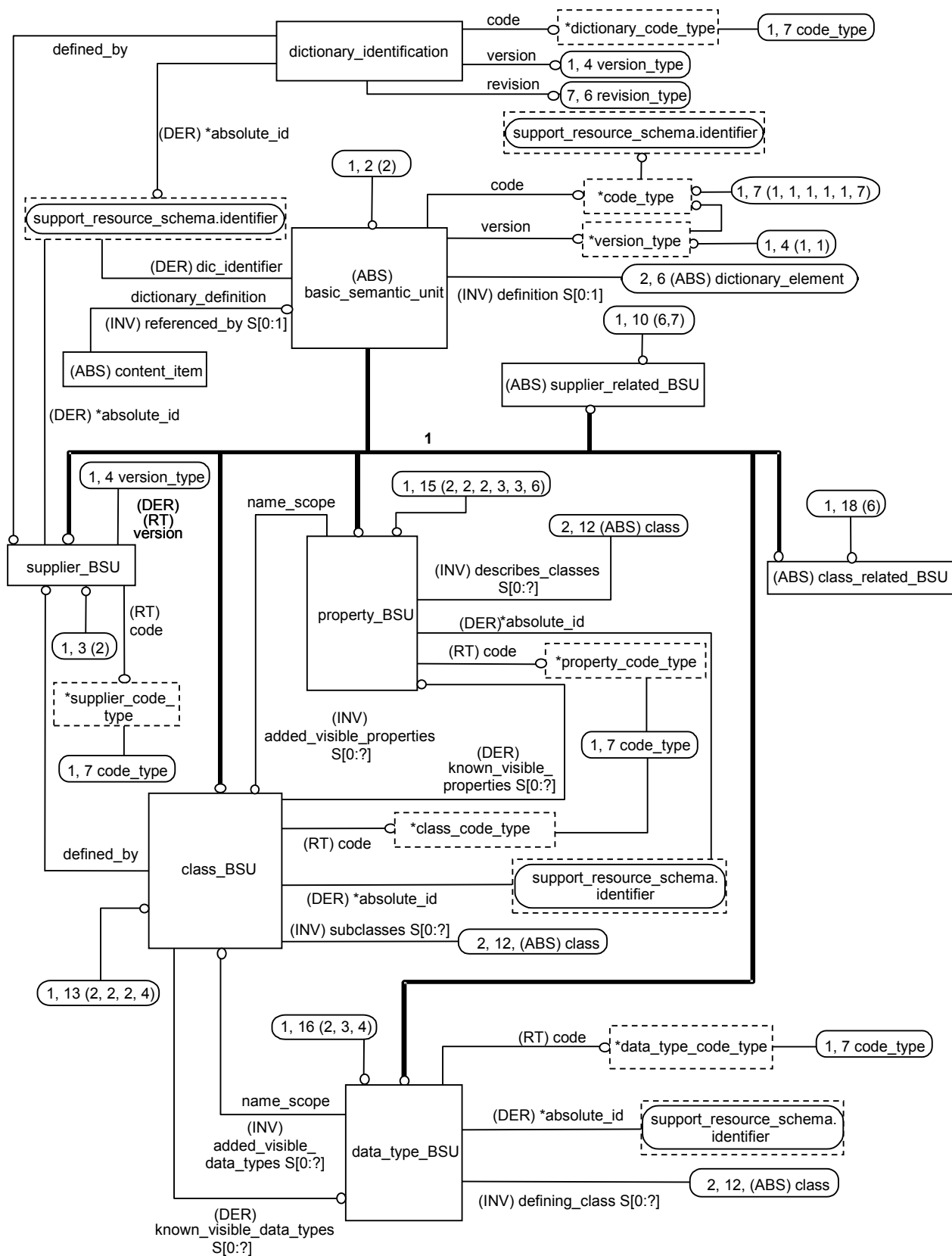


Figure B.1 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 1 sur 7

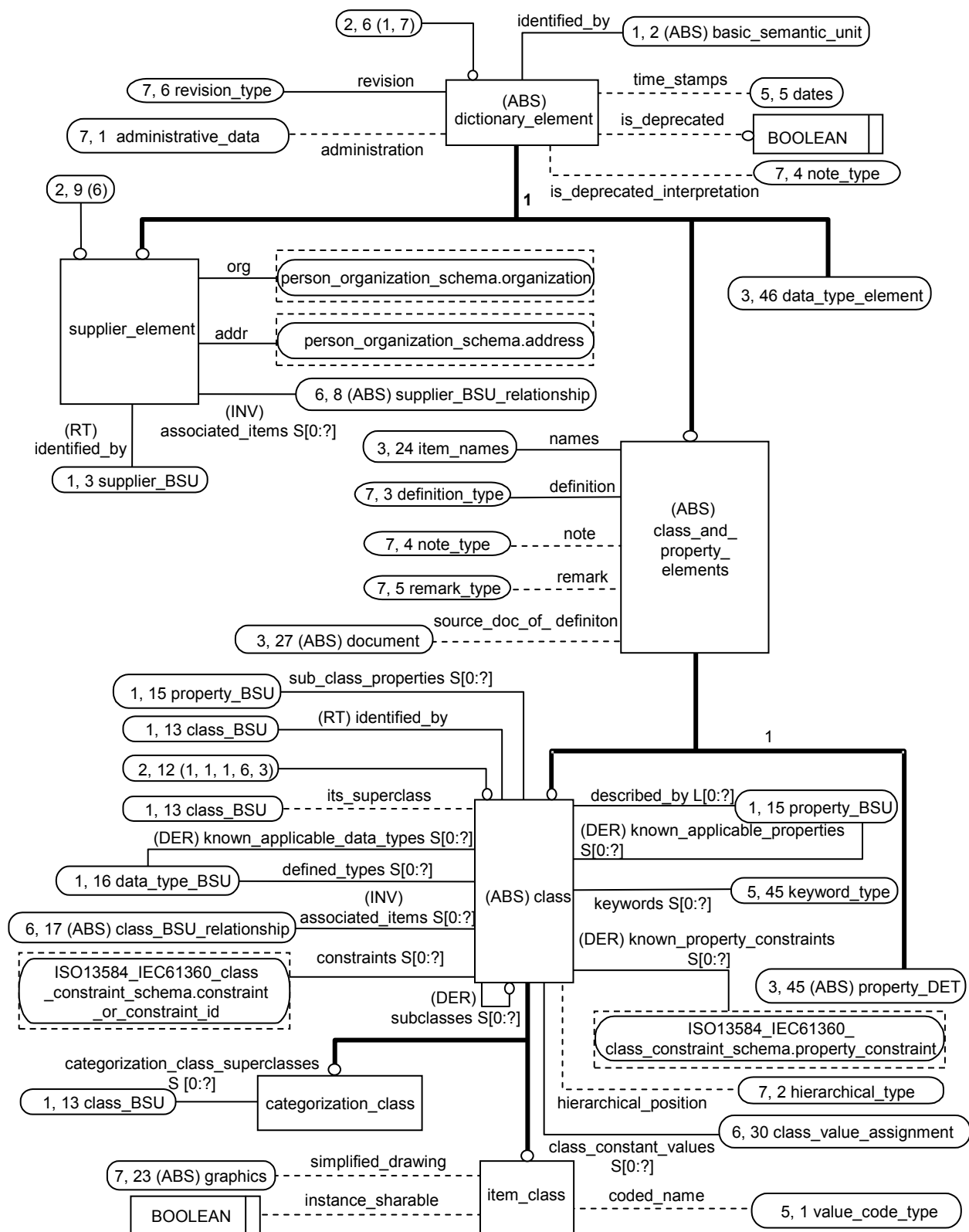


Figure B.2 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 2 sur 7



COPYRIGHT © IEC. NOT FOR COMMERCIAL USE OR REPRODUCTION

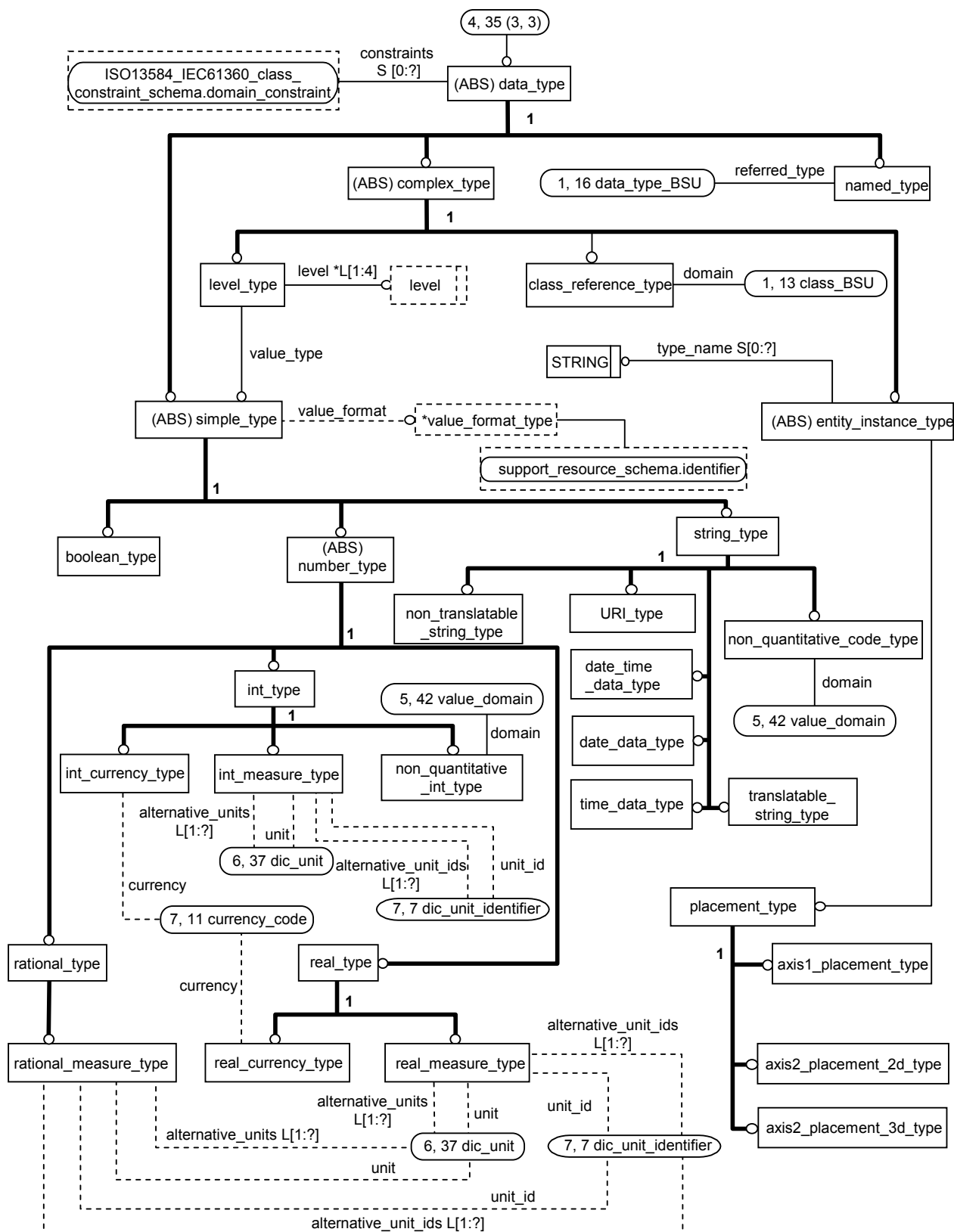


Figure B.4 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 4 sur 7



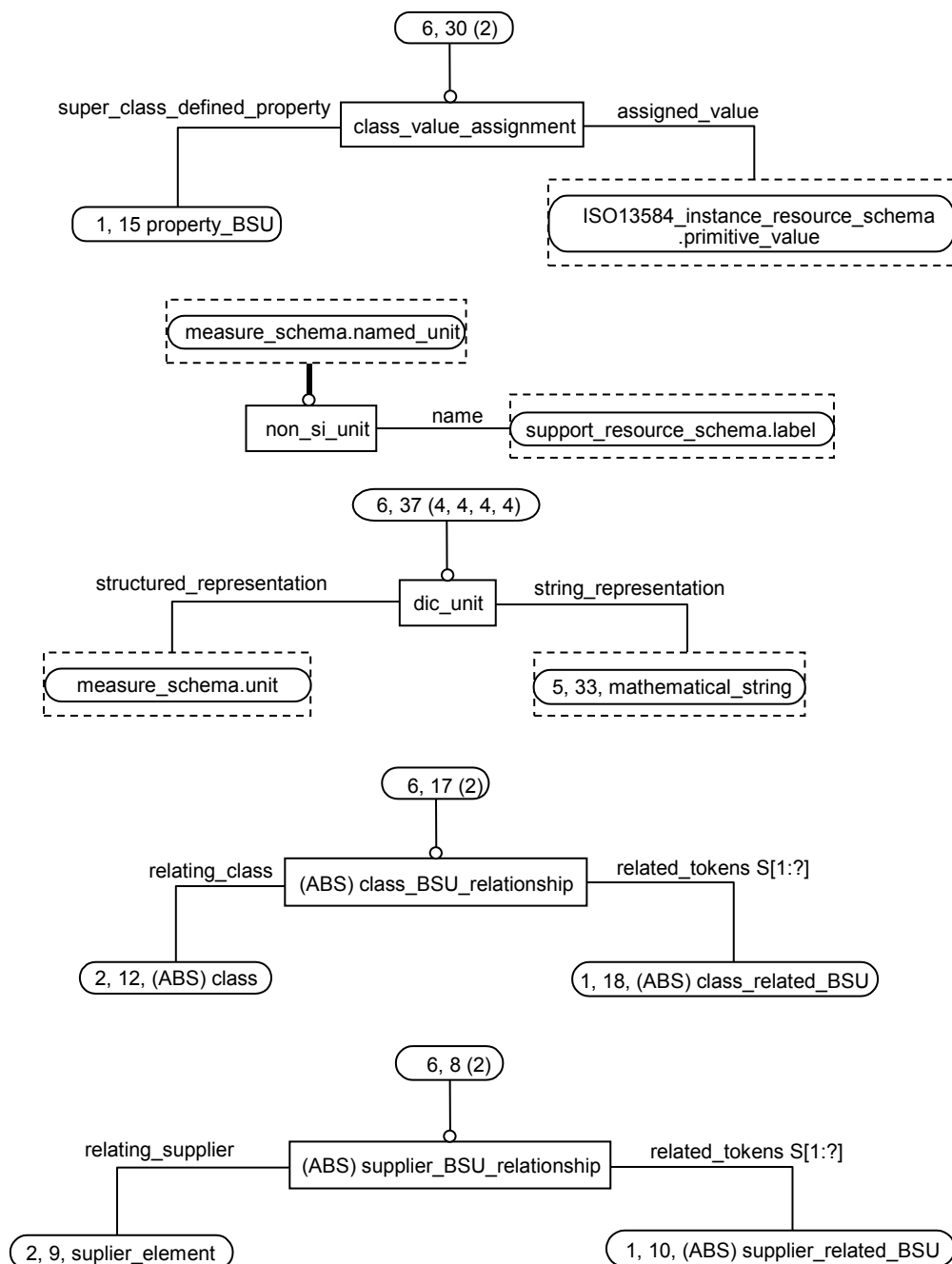


Figure B.6 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 6 sur 7

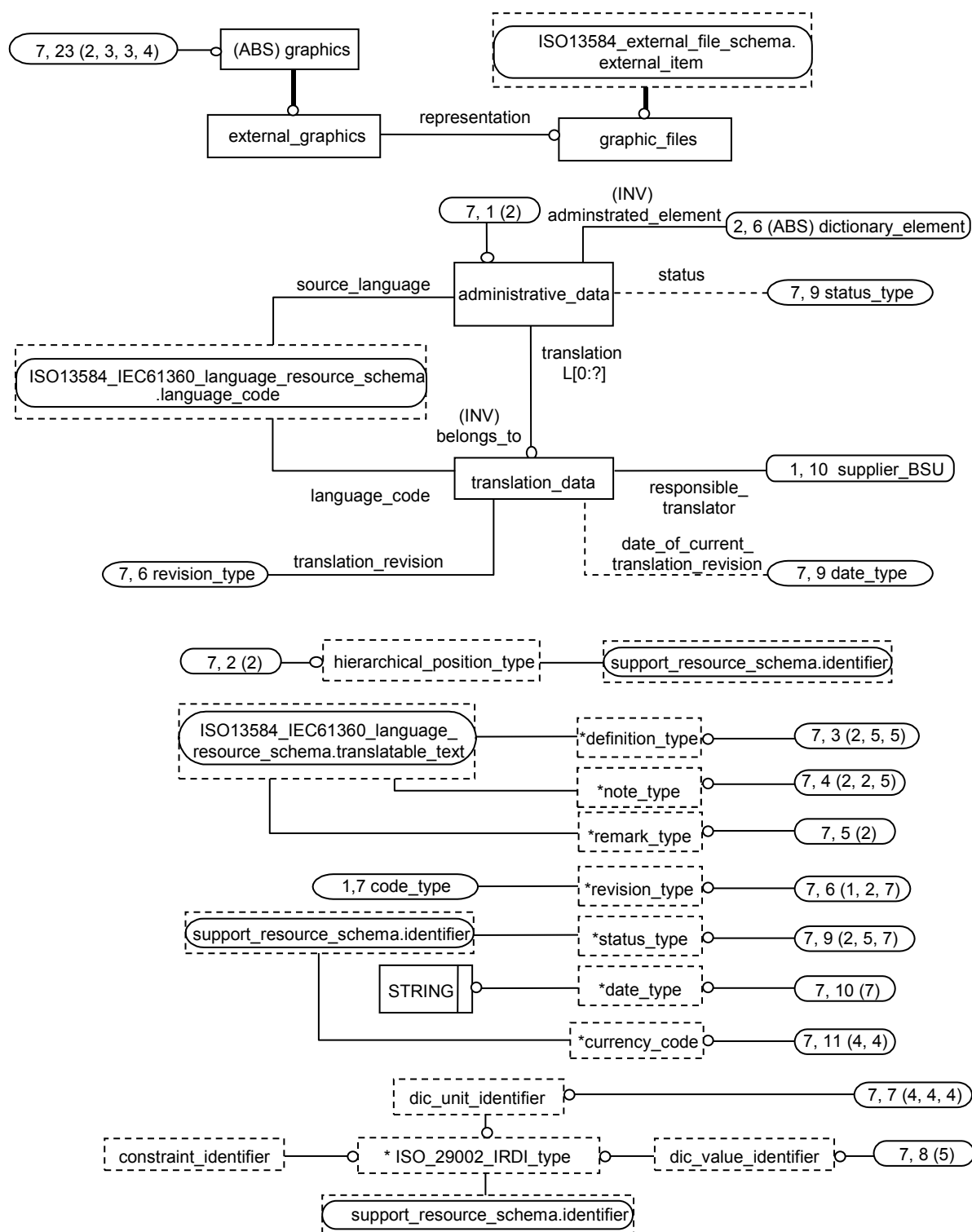
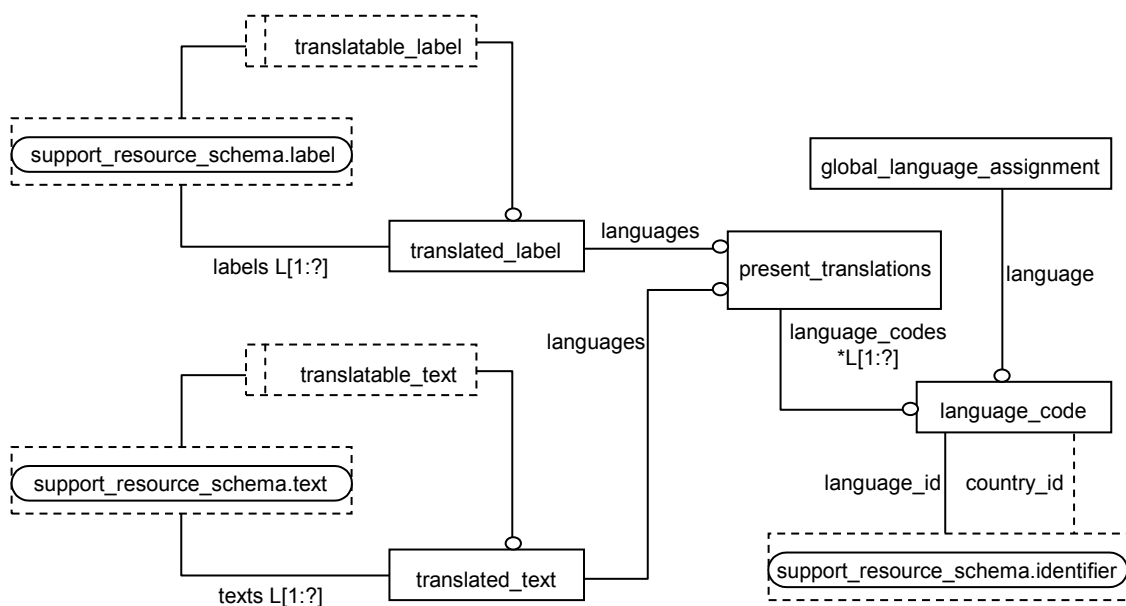


Figure B.7 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 7 sur 7



**Figure B.8 – ISO13584_IEC61360_language_resource_schema –
Diagramme EXPRESS-G 1 sur 1**

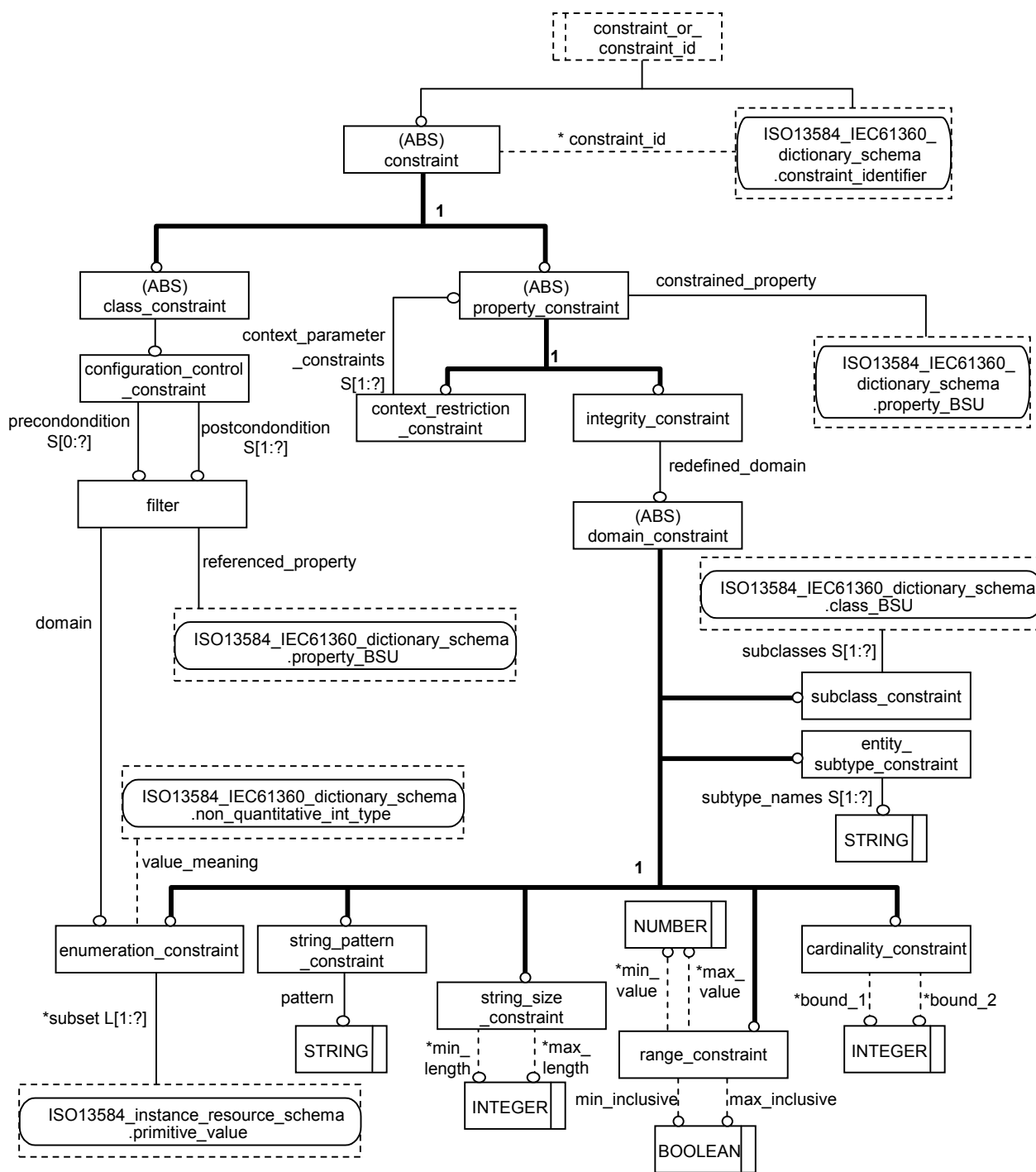


Figure B.9 – ISO13584_IEC61360_constraint_schema – Diagramme EXPRESS-G 1 sur 1

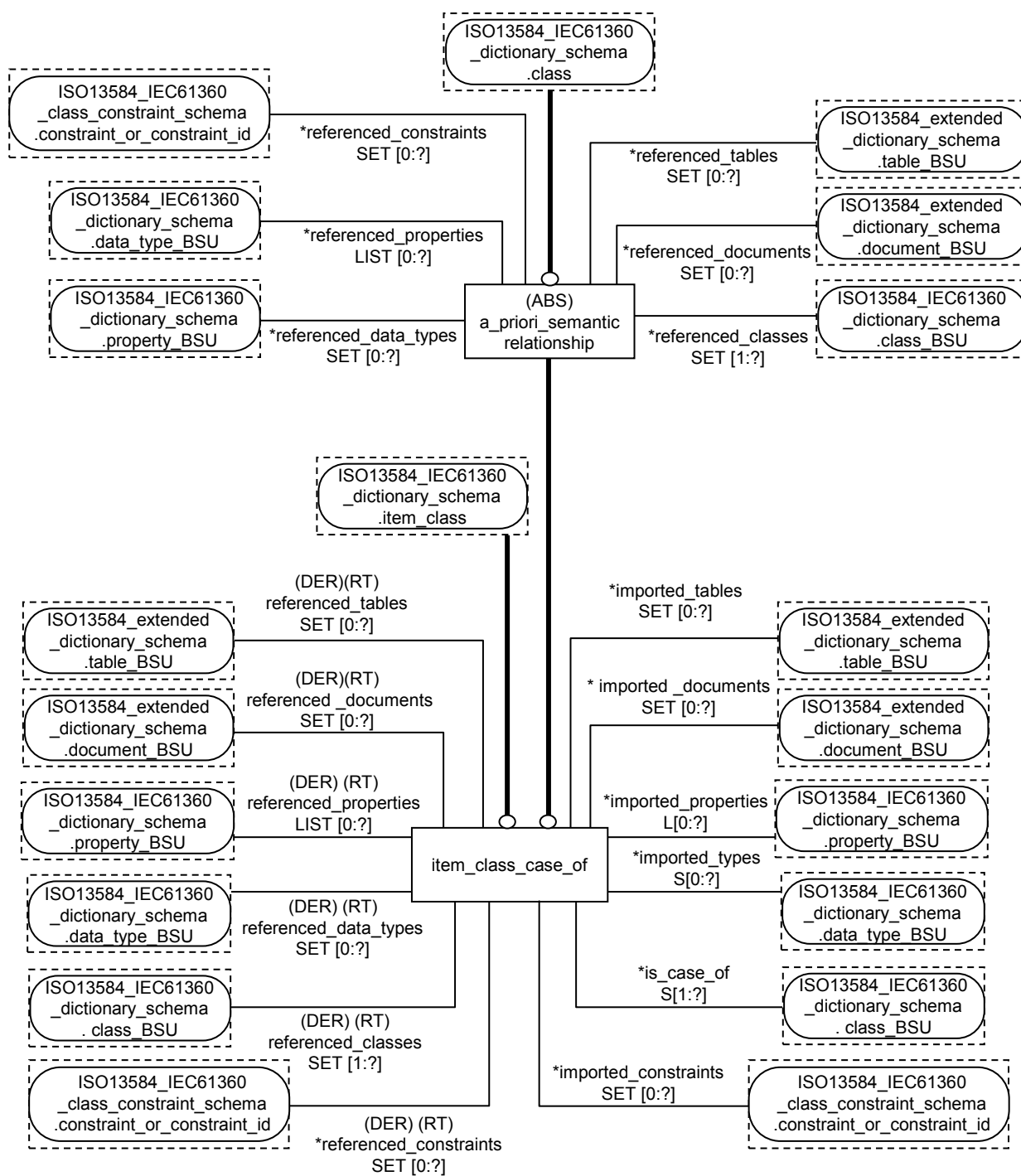


Figure B.10 – ISO13584_IEC61360_item_class_case_of_schema –
Diagramme EXPRESS-G 1 sur 1

Annexe C (informative)

Dictionnaires partiels

Le modèle de données EXPRESS publié dans la présente partie de la CEI 61360 et dupliqué pour la commodité dans l'ISO 13584-42. Il permet de décrire des dictionnaires composés de classes, de propriétés et de types de données et il permet la prise en charge de leur identification unique par le biais du mécanisme des unités sémantiques de base (BSU). Ce modèle permet de décrire des hiérarchies de classes structurées selon une structure arborescente utilisant un mécanisme d'héritage simple. Avec ce modèle de données, seuls les dictionnaires uniques et autonomes étaient traités.

L'utilisation de ce modèle de données conduit à plusieurs dictionnaires différents. Lors du processus de construction d'un dictionnaire, il peut arriver que des concepteurs exigent de référencer une classe particulière, une propriété particulière ou un type de données particulier déjà défini(e) dans un autre dictionnaire. La possibilité d'importer des propriétés et des types de données dans le dictionnaire en cours de conception est offerte par la relation "case-of" qui peut être utilisée avec le modèle complet de dictionnaire commun ISO13584/CEI61360, documenté tant dans l'ISO 13584-25 que dans la CEI 61360-5. En fait, cette relation permet d'importer des propriétés et des types de données définies extérieurement donnant la possibilité de prendre en charge des dictionnaires partiels et d'éviter la duplication entre dictionnaires, chaque dictionnaire définissant sa propre structure de classes.

Le modèle complet de dictionnaire commun ISO13584/CEI61360 propose deux mécanismes:

- l'entité EXPRESS **a_priori_case_of_semantic_relationship** permet d'utiliser directement les propriétés ou les types de données défini(e)s dans un ou plusieurs dictionnaires externes sans les décrire de nouveau;
- l'entité EXPRESS **a_posteriori_case_of_relationship** permet, une fois les propriétés ou les types de données déjà défini(e)s dans un dictionnaire en cours de conception, de les mettre en correspondance avec des propriétés ou types de données correspondant(e)s défini(e)s dans un dictionnaire externe.

Ces mécanismes permettent de concevoir des dictionnaires qui se réfèrent à des éléments de données qui sont définis dans d'autres dictionnaires sans affecter leur signification sémantique. En outre, les relations "case-of" sont enregistrées dans les dictionnaires qui utilisent cette fonctionnalité, permettant la prise en charge de l'intégration automatique de dictionnaires basés sur les mêmes dictionnaires normalisés.

Ce mécanisme est également recommandé lors de la conception d'un dictionnaire utilisateur final. En règle générale, un dictionnaire utilisateur final n'a pas besoin de toute la structure de classes définie dans les dictionnaires normalisés, tout en souhaitant pouvoir échanger de l'information avec d'autres utilisateurs dont les dictionnaires sont basés sur le(s) même(s) dictionnaire(s) normalisé(s). Si l'utilisateur final définit sa propre hiérarchie, mais met en correspondance chacune de ses classes, par la relation "case-of", avec la classe normalisée correspondante et s'il/elle importe toutes les propriétés normalisées existantes qui s'avèrent utiles pour son contexte, tout en ajoutant des propriétés spécifiques à un utilisateur, l'utilisateur personnalise son propre dictionnaire tout en pouvant échanger des informations normalisées avec d'autres utilisateurs. Cette approche de la conception de dictionnaires utilisateur final est celle recommandée dans la présente norme.

Annexe D (normative)

Spécification du format des valeurs

D.1 Généralités

La présente partie de la CEI 61360 et l'ISO 13584-42 fournissent une syntaxe particulière pour spécifier les formats autorisés pour les valeurs de chaîne et les valeurs numériques qui peuvent être associées à une propriété.

EXEMPLE 1 Le format NR1 3 permet de spécifier que seules des valeurs entières constituées de trois chiffres exactement sont autorisées.

NOTE 1 Aucun format de valeur n'est défini pour un quelconque autre **data_type**, y compris le **boolean_type**.

NOTE 2 Dans la présente partie de la CEI 61360, définir le format des valeurs de propriété n'est pas obligatoire.

La syntaxe des formats autorisés est définie par la forme de Backus-Naur étendue (EBNF pour «Extended Backus-Naur Form») définie dans l'ISO/CEI 14977.

EXEMPLE 2 La syntaxe du format NR1 3 est constituée des lettres 'NR1' ' ' '3'.

La signification de chaque syntaxe, à savoir les caractères qui peuvent être utilisés pour représenter une valeur, ne peut pas être définie à l'aide de la forme EBNF. Donc, la signification de chaque partie du format concernant les caractères autorisés pour représenter la valeur est spécifiée séparément pour chaque partie du format.

EXEMPLE 3 La syntaxe du format NR1 3 a la signification suivante: *NR1* signifie que seule une valeur entière (integer) peut être représentée. L'espace signifie qu'un nombre fixe de caractères est spécifié par le format. 3 signifie que trois chiffres exactement sont requis.

D.2 Notation

Le Tableau D.1 résume le sous-ensemble du métalangage syntaxique EBNF de l'ISO/CEI 14977 utilisé par la présente partie de la CEI 61360 pour spécifier le format des valeurs des propriétés.

En utilisant ces notations, la syntaxe du sous-ensemble du métalangage EBNF utilisée par la présente partie de la CEI 61360 pour spécifier le format de valeurs des propriétés est récapitulée par la grammaire suivante (le caractère, la lettre et le chiffre du meta-identifiant («méta-identificateur») ne sont pas détaillés):

```
syntax = syntaxrule, { syntaxrule } ;
syntaxrule = metaidentifier, '=', definitionslist, ';' ;
definitionslist = singledefinition, { '|', singledefinition } ;
singledefinition = term, { ',', term } ;
term = primary, [ '-', primary ] ;
primary = optionalsequence | repeatedsequence | groupedsequence |
        metaidentifier | terminal | empty ;
optionalsequence = '[' definitionslist ']' ;
repeatedsequence = '{' definitionslist '}' ;
groupedsequence = '(' definitionslist ')' ;
metaidentifier = letter, { letter } ;
terminal = '"', (character - '"'), {character - '"'}, '"'
        | "'", (character - "'"), {character - "'"}, "'" ;
empty = ;
```

Le signe '=' indique une règle de syntaxe. Le meta-identifiant sur la gauche peut être réécrit par la combinaison des éléments situés à droite. Tous les espaces éventuels apparaissant entre les éléments sont sans signification, sauf s'ils apparaissent au sein d'un `terminal`. Une règle de syntaxe se termine par un point-virgule ';'.

Tableau D.1 – Métalangage syntaxique EBNF de l'ISO/CEI 14977

Représentation	Noms de caractère ISO/CEI 10646-1	Symbole de métalangage et rôle
' '	apostrophe	Premier symbole de citation: représente les bornes du langage. Le texte ne doit pas contenir d'apostrophe. Exemple: 'Hello'
" "	guillemets de citation	Second symbole de citation: représente les bornes du langage. Le texte ne doit pas contenir de guillemets. Exemple: "Voiture de Jean"
()	parenthèse ouvrante, parenthèse fermante	Début / fin de symboles de groupe. Le contenu est considéré comme étant un seul symbole.
[]	crochet ouvrant, crochet fermant	Début / fin de symboles d'option. Le contenu peut ou peut ne pas être présent.
{ }	accolade ouvrante, accolade fermante	Début / fin de symboles de répétition. Le contenu peut être présent 0 fois à n fois.
-	trait d'union-moins	Symbole d'exception
,	virgule	symbole de concaténation
=	signe égal	Symbole de définition. Règle de syntaxe: définit le symbole de gauche par la formule située à droite.
	trait vertical	Symbole séparateur d'alternatives
;	point-virgule	Symbole de fin Fin d'une règle de syntaxe.

L'utilisation d'un méta-identificateur dans une liste de définition désigne un symbole non borné qui apparaît sur la gauche d'une autre règle de syntaxe. Un méta-identificateur se compose de lettres ou de chiffres, le premier caractère étant une lettre. Si un terme contient à la fois un `primary` («primaire») précédant un signe moins, et un `primary` qui suit un signe moins, seule la séquence de symboles qui sont représentés par le premier `primary` et qui ne sont pas représentés par le second `primary` sont représentés par le terme.

EXEMPLE 1 Notation:

""', caractère – ""', ""'

signifie n'importe quel caractère sauf le caractère apostrophe, inséré entre deux caractères apostrophes.

Le `terminal` désigne un symbole qui ne peut pas être étendu davantage par une règle syntaxique, et qui apparaîtra dans le résultat final. Deux façons sont permises pour représenter un `terminal`: soit un ensemble de caractères sans apostrophe, inséré entre

deux apostrophes, soit un ensemble de caractères sans guillemets, inséré entre deux guillemets.

EXEMPLE 2 Supposons que nous voulons décrire, par une telle grammaire, le prix d'un produit en €. Un tel prix est un nombre positif avec pas plus de 2 chiffres dans la partie des centimes. Nous introduisons trois méta-identificateurs associés à trois règles de syntaxe:

```
chiffre = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';
centimes = [ '.', chiffre [, chiffre] ];
euros = chiffre {, chiffre} centimes;
```

Avec ces règles de syntaxe: 012, 4323.3, 3.56 sont des exemples de représentations légitimes des Euros. 12.,.10 sont des exemples de représentations illégitimes des euros.

D.3 Types de format de valeurs de données

La grammaire définie la présente annexe définit huit différents types de formats de valeurs: quatre formats de valeurs quantitatives et cinq formats de valeurs non quantitatives.

Dans le prochain article, nous définissons les méta-identificateurs qui sont utilisés pour spécifier ces formats. À l'Article D.5, nous définissons la règle de syntaxe pour les quatre méta-identificateurs qui représentent les quatre formats de valeurs quantitatives, accompagnés de leur signification au niveau de la valeur. À l'Article D.6 nous définissons les méta-identificateurs pour les cinq formats de valeurs non quantitatives, accompagnés de leur signification au niveau de la valeur.

D.4 Méta-identificateur utilisé pour définir les formats

Les méta-identificateurs utilisés dans la grammaire qui définissent les divers formats de valeurs sont les suivants:

dot = '.';

decimalMark = '.';

exponentIndicator = 'E';

numeratorIndicator = 'N';

denominatorIndicator = 'D';

leadingDigit = '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';

lengthOfExponent = leadingDigit, {trailingDigit};

lengthOfIntegerPart = (leadingDigit, {trailingDigit});

lengthOfNumerator = leadingDigit, {trailingDigit};

lengthOfDenominator = leadingDigit, {trailingDigit};

lengthOfFractionalPart = (leadingDigit, {trailingDigit}) | '0' ;

lengthOfIntegralPart = (leadingDigit, {trailingDigit}) | '0' ;

lengthOfNumber = leadingDigit, {trailingDigit};

`trailingDigit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';`

`signedExponent = 'S';`

`signedNumber = space, 'S';`

`space = ' ';`

`variableLengthIndicator = '..';`

`decimalMark`: séparateur entre partie entière et partie fractionnaire de nombres au format NR2 ou NR3.

`leadingDigit`: premier chiffre d'un nombre comprenant un ou plusieurs chiffres.

`trailingDigit`: un des chiffres qui se combinent pour former des nombres, sauf le premier.

NOTE Si un nombre comprend un seul chiffre, aucun `trailingDigit` n'est présent.

D.5 Formats de valeurs quantitatives

D.5.1 Généralités

Les quatre règles de syntaxe du format de valeurs quantitatives et leurs significations pour la représentation des valeurs sont définies dans les quatre paragraphes suivants. Elles sont autorisées à l'emploi pour des propriétés ayant les types de données suivants:

- **number_type** ou n'importe lequel de ses sous-types;
- **level_type** dont le **value_type** est soit **real_measure_type**, soit **int_measure_type**;
- **list_type**, **set_type**, **bag_type**, **array_type** ou **set_with_subset_constraint_type** dont le **value_type** est **number_type** ou n'importe lequel de ses sous-types.

NOTE 1 **list_type**, **set_type**, **bag_type**, **array_type** ou **set_with_subset_constraint_type** sont définis dans l'ISO 13584-25.

NOTE 2 Pour **non_quantitative_int_type**, le format de valeurs s'applique au code.

NOTE 3 Il convient que la valeur de cet attribut soit compatible avec le type de données de la propriété: il convient qu'elle ne change pas ce type de données, sinon il convient de l'ignorer.

EXEMPLE Le format de valeurs NR2 n'est pas compatible avec **int_type**, car il convient que les valeurs entières n'aient pas de partie fractionnaire.

D.5.2 Format de valeurs NR1

La syntaxe des valeurs NR1 spécifie le format d'une valeur d'une propriété entière.

Règle de syntaxe:

```
NR1Value = 'NR1', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
lengthOfNumber;
```

La signification des composants du format de valeurs NR1 pour la représentation de valeurs est comme suit:

- **'NR1'**: la valeur doit être un nombre entier.

NOTE 1 Il convient que les valeurs des nombres NR1 ne contiennent aucun espace.

- `lengthOfNumber`: nombre de chiffres de la valeur.

NOTE 2 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres peut être inférieur.

- `signedNumber`: si `signedNumber` est présent, le nombre correspondant doit avoir une valeur positive, négative ou nulle (zéro). En cas de valeurs positives, un signe '+' peut être présent. Les valeurs négatives doivent être précédées d'un signe '-'. La valeur zéro ne doit pas être précédée d'un signe '-'.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, le nombre correspondant doit contenir un nombre de chiffres qui est inférieur ou égal à sa spécification de longueur, c'est-à-dire, à `lengthOfNumber`.

D.5.3 Format des valeurs NR2

La syntaxe des valeurs NR2 spécifie le format d'une valeur de propriété réelle qui n'a pas besoin d'un exposant.

Règle de syntaxe:

```
NR2Value = 'NR2', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
lengthOfIntegralPart, decimalMark, lengthOfFractionalPart;
```

La signification des composants du format de valeurs NR2 pour la représentation de valeurs est comme suit:

- 'NR2': la valeur doit être un réel.

NOTE 1 Il convient que les valeurs des nombres NR2 ne contiennent aucun espace.

- `lengthOfFractionalPart`: nombre de chiffres de la partie fractionnaire du nombre.

NOTE 2 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de la partie fractionnaire peut être inférieur.

NOTE 3 `lengthOfFractionalPart` spécifie de façon implicite l'exactitude recommandée de la valeur. L'exactitude effective du nombre à partir duquel cette valeur a été dérivée peut avoir été plus élevée que la valeur exprimée ici.

- `lengthOfIntegralPart`: nombre de chiffres de la partie entière du nombre.

NOTE 4 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de la partie entière peut être inférieur.

- `signedNumber`: si `signedNumber` est présent, le nombre correspondant doit avoir une valeur positive, négative ou nulle (zéro). En cas de valeurs positives, un signe '+' peut être présent. Les valeurs négatives doivent être précédées d'un signe '-'. La valeur zéro ne doit pas être précédée d'un signe '-'.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, soit la partie entière, soit la partie fractionnaire du nombre, soit les deux parties, doit(ven)t contenir un nombre de chiffres qui est inférieur ou égal à sa spécification de longueur correspondante, c'est-à-dire, à `lengthOfIntegralPart` ou à `lengthOfFractionalPart`. Au moins un chiffre doit être présent dans le nombre.

D.5.4 Format de valeurs NR3

La syntaxe des valeurs NR3 spécifie le format d'une valeur d'une propriété réelle qui est représentée avec un exposant.

Règle de syntaxe:

```
NR3Value = 'NR3', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
```

`lengthOfIntegralPart, decimalMark, lengthOfFractionalPart, exponentIndicator, [signedExponent], lengthOfExponent;`

La signification des composants du format de valeurs NR3 pour la représentation de valeurs est comme suit:

- 'NR3': la valeur doit être un réel avec un exposant de base 10.

NOTE 1 Il convient qu'il y ait au moins un chiffre et le signe décimal dans la mantisse. L'exposant doit contenir au moins un chiffre, aussi.

NOTE 2 Les valeurs des nombres NR3 ne doivent contenir aucun espace.

- `exponentIndicator`: séparateur entre mantisse et exposant dans les nombres au format NR3.
- `lengthOfExponent`: nombre de chiffres de l'exposant.

NOTE 3 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de l'exposant peut être inférieur.

NOTE 4 Les signes existants éventuellement ou un signe décimal ne sont pas comptés par `lengthOfNumber`, `lengthOfIntegralPart`, `lengthOfFractionalPart` ou `lengthOfExponent`.

- `lengthOfFractionalPart`: nombre de chiffres de la partie fractionnaire de la mantisse.

NOTE 5 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de la partie fractionnaire peut être inférieur.

NOTE 6 `lengthOfFractionalPart` spécifie de façon implicite l'exactitude recommandée d'une valeur. L'exactitude effective du nombre à partir duquel cette valeur a été dérivée peut avoir été plus élevée que la valeur exprimée ici.

- `lengthOfIntegralPart`: nombre de chiffres de la partie entière de la mantisse.

NOTE 7 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de la partie entière peut être inférieur.

- `signedExponent`: si `signedExponent` est présent, l'exposant correspondant doit avoir une valeur positive, négative ou nulle (zéro). En cas de valeurs positives, un signe '+' peut être présent. Les valeurs négatives doivent être précédées d'un signe '-'. La valeur zéro ne doit pas être précédée d'un signe '-'.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, le nombre ou exposant correspondant doit contenir un nombre de chiffres qui est inférieur ou égal à sa spécification de longueur correspondante, c'est-à-dire, à `lengthOfIntegralPart`, à `lengthOfFractionalPart` ou à `lengthOfExponent`. Au moins un chiffre doit être présent dans la mantisse et dans l'exposant.

D.5.5 Format de valeurs NR4

La syntaxe de valeurs NR4 spécifie le format d'une valeur de propriété rationnelle qui est représentée avec une partie entière, et éventuellement une partie fractionnaire avec un dénominateur et un numérateur.

Règle de syntaxe:

`NR4Value = 'NR4', ((signedNumber, variableLengthIndicator) | (signedNumber, space) | variableLengthIndicator | space), lengthOfIntegerPart, numeratorIndicator, lengthOfNumerator, denominatorIndicator, lengthOfDenominator;`

La signification des composants du format de valeurs NR4 pour la représentation de valeurs est comme suit:

- 'NR4': la valeur doit être un nombre rationnel représenté soit comme un nombre entier, soit comme une fraction constituée d'un numérateur et d'un dénominateur, soit comme un entier et une fraction.

EXEMPLE 12 ½ et 12 ¾ sont des valeurs qui peuvent être représentées dans le format NR4.

NOTE 1 Il doit y avoir au moins un chiffre soit dans la partie entière, soit à la fois dans la partie numérateur et dans la partie dénominateur. Si une partie d'une fraction contient un chiffre, l'autre partie doit aussi contenir un certain nombre de chiffres. Toutes les trois parties peuvent aussi contenir des chiffres.

NOTE 2 Les valeurs des nombres NR4 ne doivent contenir aucun espace.

- `numeratorIndicator`: séparateur entre la description de la partie entière et la description de la partie fraction dans les formats NR4.
- `lengthOfNumerator`: nombre de chiffres du numérateur.

NOTE 3 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres du numérateur peut être inférieur.

NOTE 4 Si la valeur du nombre rationnel est complètement représentée par sa partie entière, ni le numérateur de la fraction ni son dénominateur ne doivent être représentés.

- `denominatorIndicator`: séparateur entre la description de la partie numérateur et la description de la partie dénominateur dans les formats NR4.
- `lengthOfDenominator`: nombre de chiffres du dénominateur.

NOTE 5 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres du dénominateur peut être inférieur.

NOTE 6 Si la valeur du nombre rationnel est complètement représentée par sa partie entière, ni le numérateur de la fraction ni son dénominateur ne doivent être représentés.

- `lengthOfIntegerPart`: nombre de chiffres de la partie entière du nombre rationnel.

NOTE 7 S'il est précédé d'un `variableLengthIndicator`, le nombre effectif de chiffres de la partie entière peut être inférieur.

- `variableLengthIndicator`: si `variableLengthIndicator` est présent, les trois parties du nombre rationnel doivent contenir un nombre de chiffres qui est inférieur ou égal à sa spécification de longueur correspondante, c'est-à-dire, à `lengthOfIntegralPart`, à `lengthOfNumerator` ou à `lengthOfDenominator`. Au moins un chiffre doit être présent soit dans la partie entière, soit dans les deux parties de la fraction.

D.6 Formats de valeurs non quantitatives

D.6.1 Généralités

Les règles de syntaxe du format des valeurs non quantitatives et leurs significations sont définies dans les cinq paragraphes suivants. Elles sont autorisées à l'emploi pour des propriétés ayant les types de données suivants:

- **`string_type`** ou importe lequel de ses sous-types;
- **`list_type`, `set_type`, `bag_type`, `array_type` ou `set_with_subset_constraint_type`** dont le **`value_type`** est **`string_type`** ou n'importe lequel de ses sous-types.

NOTE 1 **`list_type`, `set_type`, `bag_type`, `array_type` ou `set_with_subset_constraint_type`** sont définis dans l'ISO 13584-25.

NOTE 2 Pour **`translatable_string_type`**, le format de valeurs s'applique à n'importe quelle représentation spécifique à un langage de la chaîne de caractères.

NOTE 3 Pour **`non_quantitative_code_type`**, le format de valeurs s'applique au code.

Les valeurs non quantitatives sont représentées par des chaînes qui comprennent des caractères. La longueur d'une chaîne peut être spécifiée soit en spécifiant directement la

limite supérieure du nombre de caractères contenus, soit en spécifiant que le nombre total de caractères peut être tout multiple entier de la longueur spécifiée.

Règle de syntaxe:

`factor` = `leadingDigit`, {`trailingDigit`};

`numberOfCharacters` = (`leadingDigit`, {`trailingDigit`})('nx', `factor`,');)

La signification des composants `factor` («facteur») est comme suit

- `factor`: lorsque `factor` est présent, `numberOfCharacters` doit être un multiple entier de la valeur donnée dans `factor`. `Factor` ne doit pas contenir la valeur zéro.
- `numberOfCharacters`: détermine la quantité maximale des caractères contenus dans la chaîne.

D.6.2 Format de valeurs alphabétiques

Un “Alphabetic Value Format ((A) «Format de valeurs alphabétiques»)” définit le format de valeurs d'une chaîne de caractères qui contient des lettres alphabétiques. Donc, le contenu doit être pris dans les caractères de la rangée 00, cellule 20, cellule 40 à 7E, ou cellule C0 à FF, du Basic Multilingual Plane ((BMP) «Plan multilingue de base») (Plan 00 du Groupe 00) de l'ISO/CEI 10646-1.

NOTE 4 En raison des problèmes potentiels d'interprétation du contenu des valeurs au sein de composants d'un seul système ou de plusieurs systèmes, la recommandation est que, lorsque cela est possible, il convient de limiter les caractères utilisés au jeu G0 de l'ISO/CEI 10646-1 et/ou de la rangée 00 colonnes 002 à 007 de l'ISO/CEI 10646-1.

NOTE 5 Pour les autres langues supportées par les données de traduction, les caractères ou idéogrammes pertinents du jeu correspondant de caractères spécifique à une langue sont disponibles tels que définis par la norme Unicode. Dans la plupart des cas, il n'y aura pas de relation du type 1:1 (biunivoque) entre les caractères de la langue source aux caractères de la langue cible.

EXEMPLE Idéogrammes CJC (Chinois – japonais – coréen).

Règle de syntaxe:

`AValue` = 'A', (space | `variableLengthIndicator`), `numberOfCharacters`;

La signification des composants du format de valeurs A pour la représentation de valeurs est comme suit:

- 'A': la valeur doit être une chaîne, ou plusieurs sous-chaînes, de lettres alphabétiques.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, la chaîne peut contenir moins de caractères qu'il n'en est indiqué par `numberOfCharacters`. La chaîne doit contenir au moins un caractère.

D.6.3 Format de valeurs en caractères mélangés

Un format “Mixed value format («format de valeurs mélangées»(M))” définit le format de valeurs d'une chaîne qui peut contenir n'importe quel caractère spécifié à l'Article D.7.

NOTE Pour les autres langues supportées par les données de traduction, les caractères ou idéogrammes pertinents du jeu correspondant de caractères spécifique à une langue sont disponibles tels que définis par la norme Unicode.

EXEMPLE Caractères CJC (Chinois – japonais – coréen).

Règle de syntaxe:

`MValue` = 'M', (space | `variableLengthIndicator`), `numberOfCharacters`;

La signification des composants du format de valeurs M pour la représentation de valeurs est comme suit:

- 'M': la valeur doit être une chaîne, ou plusieurs sous-chaînes.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, la chaîne peut contenir moins de caractères qu'il n'en est indiqué par `numberOfCharacters`. La chaîne doit contenir au moins un caractère.

D.6.4 Format de valeurs numériques

Un "Number value format ((N) «format de valeurs numériques»)" définit le format de valeurs d'une chaîne de caractères qui contient des chiffres numériques seulement. Donc, le contenu doit être pris dans les caractères de la rangée 00, cellule 2B, cellule 2D, cellule 30 à 39, ou cellule 45, du Basic Multilingual Plane ((BMP) «plan multilingue de base» (Plan 00 du Groupe 00) de l'ISO/CEI 10646-1.

NOTE Pour les autres langues supportées par les données de traduction, les caractères ou idéogrammes pertinents du jeu correspondant de caractères spécifiques à une langue sont disponibles tels que définis par la norme Unicode.

EXEMPLE Le Tableau D.2 montre la transposition des chiffres européens "0" à "9" en chiffres arabes.

Tableau D.2 – Transposition des chiffres de style européen en chiffres arabes

Chiffres européens	9	8	7	6	5	4	3	2	1	0
Chiffres arabes	٩	٨	٧	٦	٥	٤	٣	٢	١	٠

Règle de syntaxe:

NValue = 'N', (space | `variableLengthIndicator`), `numberOfCharacters`;

La signification des composants du format de valeurs N pour la représentation des valeurs est comme suit:

- 'N': la valeur doit être une chaîne, ou plusieurs sous-chaînes, de chiffres numériques.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, la chaîne peut contenir moins de caractères qu'il n'en est indiqué par `numberOfCharacters`. La chaîne doit contenir au moins un caractère.

D.6.5 Format de valeurs en caractères numériques ou alphabétiques mélangés

Un "Mixed alphabetic or numeric characters value format («Format de valeurs en caractères numériques ou alphabétiques mélangés» (X))" définit le format des valeurs d'une chaîne qui contient des caractères alphanumériques, à savoir toute combinaison de caractères issus d'un format de valeurs A ou d'un format de valeurs N.

NOTE Pour les autres langues supportées par les données de traduction, les caractères ou idéogrammes pertinents du jeu correspondant de caractères spécifique à une langue sont disponibles tels que définis par la norme Unicode.

Règle de syntaxe:

XValue = 'X', (space | `variableLengthIndicator`), `numberOfCharacters`;

La signification des composants du format de valeurs X pour la représentation de valeurs est comme suit:

- 'X': la valeur doit être une chaîne, ou plusieurs sous-chaînes, de caractères alphanumériques, à savoir toute combinaison de caractères alphabétiques et numériques;

- `variableLengthIndicator`: si `variableLengthIndicator` est présent, la chaîne peut contenir moins de caractères qu'il n'en est indiqué par `numberOfCharacters`. La chaîne doit contenir au moins un caractère.

D.6.6 Format de valeurs binaires

Un "Binary Value Format (Format de valeurs binaires (B))" définit le format de valeurs d'une chaîne qui contient des caractères binaires, à savoir "0" ou "1". Donc, le contenu doit être pris dans les caractères de la rangée 00, cellule 30 ou 31, du Basic Multilingual Plane ((BMP) «Plan multilingue de base») (Plan 00 du Groupe 00) de l'ISO/CEI 10646-1.

NOTE Pour les autres langues supportées par les données de traduction, les caractères ou idéogrammes pertinents du jeu correspondant de caractères spécifique à une langue sont disponibles tels que définis par la norme Unicode.

Règle de syntaxe:

`BValue = 'B', (space | variableLengthIndicator), numberOfCharacters;`

La signification des composants du format de valeurs B pour la représentation de valeurs est comme suit:

- 'B': la valeur doit être une chaîne, ou plusieurs sous-chaînes, consistant en valeurs binaires, à savoir les caractères "0" ou "1" ou leurs séquences.
- `variableLengthIndicator`: si `variableLengthIndicator` est présent, la chaîne peut contenir moins de caractères qu'il n'en est indiqué par `numberOfCharacters`. La chaîne doit contenir au moins un caractère.

D.7 Exemples de valeurs

Le Tableau D.3 ci-dessous contient certains exemples pour les valeurs qui peuvent être contenues dans des nombres et des chaînes caractérisés par le plan de formats de valeurs ci-dessus.

Tableau D.3 – Exemples de valeurs numériques

Format de la valeur	Exemples de valeurs possibles
NR1 3	123; 001; 000;
NR1..3	123; 87; 5
NR1S 3	+123; +000;
NR1S..3	-123; +1; 0; -12;
NR2 3.3	123.300; 000.400 ; 000.420;
NR2..3.3	321.233; 1.234; 23.56; 9.783;.72; 324.
NR2S 3.3	-123.123; +123.300;
NR2S..3.3	-123.123; +12.3; 0.1; +.4; -3. ; 0. ;0;
NR3 3.3E4	123.123E0004; 003.000E1000;
NR3 3.3ES4	123.123E+0004; 123.123E0004; 123,000E-0001
NR3..3.3E4	123.123E0004;.123E0001; 5.E1234
NR3S 3.3ES4	+123.123E+0004; 123.000E-0001;
NR3S..3.3ES4	-123.123E+0004; +1.00E-01;.0E0; +3.E-1; -.2E-1000;
NR4 3N2D2	001 02/03; 012 00/01; 123 03/04; 000 01/04
NR4..3N2D2	1 ½ ; 12; 123 ¾; ¼ ;
NR4S 3N2D2	+001 02/03; -012 00/01; 123 03/04; -000 01/04

Format de la valeur	Exemples de valeurs possibles
NR4S..3N2D2	-1 ½ ; 12; +123 ¾; ¼ ;
A 19	Mon nom est Reinhard, abcdefghijklmnopqrs
A..3	Abc; de; G
X..5	A23RN1; B1; ca
M..10	A23RN1; B1; ca. 256 µm;
N (nx5)	12345; 1234512345; 222223333344444;
N..(nx5)	1234; 12345; 34512345; 1234512345; 23333344444; 222223333344444; -3; 5E2;
B 1	0; 1;
B 3	011; 101;

Caractères issus de l'ISO/CEI 10646-1

Les caractères suivants doivent être utilisés pour les besoins du format Mixed value format (M) (voir D.5.3):

- tous les caractères à partir de la rangée 00 du plan multilingue de base (Basic Multilingual Plane – BMP) (Plan 00 du Groupe 00) de l'ISO/CEI 10646-1;
- les caractères des autres rangées du Basic Multilingual Plane (plan multilingue de base) de l'ISO/CEI 10646-1 tels qu'ils sont énumérés au Tableau D.4;
- pour les autres langues supportées par les données traduites, le jeu complet de caractères est disponible tel que défini par la norme Unicode.

NOTE 1 En raison des problèmes potentiels d'interprétation du contenu des valeurs au sein des composants d'un seul système ou de plusieurs systèmes, la recommandation est que, lorsque cela est possible, il convient de limiter les caractères utilisés au jeu G0 de l'ISO/CEI 10646-1 et/ou de la rangée 00 colonnes 002 à 007 de l'ISO/CEI 10646-1.

NOTE 2 La norme Unicode est publiée par The Unicode Consortium, P.O. Box 391476, Mountain View, CA 94039-1476, U.S.A., www.unicode.org.

Tableau D.4 – Caractères issus d'autres rangées du Basic Multilingual Plane (plan multilingue de base) de l'ISO/CEI 10646-1

Caractère	Nom	Rangée	Cellule
	CARON	02	C7
≡	IDENTICAL TO	22	61
∧	LOGICAL AND	22	27
∨	LOGICAL OR	22	28
∩	INTERSECTION	22	29
∪	UNION	22	2A
⊂	SUBSET OF (IS CONTAINED)	22	82
⊃	SUBSET OF (CONTAINS)	22	83
⇐	LEFTWARDS DOUBLE ARROW (IS IMPLIED BY)	21	D0
⇒	RIGHTWARDS DOUBLE ARROW (IMPLIES)	21	D2

Tableau D.4 (suite)

$\therefore$	THEREFORE	22	34
$\because$	BECAUSE	22	35
$\in$	ELEMENT OF	22	08
$\ni$	CONTAINS AS MEMBER (HAS AS AN ELEMENT)	22	0B
$\subseteq$	SUBSET OR EQUAL TO (CONTAINED AS SUB- CLASS)	22	86
$\supseteq$	SUPERSET OR EQUAL TO (CONTAINS AS SUB- CLASS)	22	87
$\int$	INTEGRAL	22	2B
$\oint$	CONTOUR INTEGRAL	22	2E
∞	INFINITY	22	1E
∇	NABLA	22	07
∂	PARTIAL DIFFERENTIAL	22	02
$\sim$	TILDE OPERATOR (DIFFERENCE BETWEEN)	22	3C
$\approx$	ALMOST EQUAL TO	22	48
$\asymp$	ASYMPTOTICALLY EQUAL TO	22	43
$\equiv$	APPROXIMATELY EQUAL TO (SIMILAR TO)	22	45
$\leq$	LESS THAN OR EQUAL TO	22	64
$\neq$	NOT EQUAL TO	22	60
$\geq$	GREATER THAN OR EQUAL TO	22	65
$\Leftrightarrow$	LEFT RIGHT DOUBLE ARROW (IF AND ONLY IF)	21	D4
$\neg$	NOT SIGN	00	AC
$\forall$	FOR ALL	22	00
$\exists$	THERE EXISTS	22	03
$\aleph$	HEBREW LETTER ALEF	05	D0
$\square$	WHITE SQUARE (D'ALEMBERTIAN OPERATOR)	25	A1
$\parallel$	PARALLEL TO	22	25
Γ	GREEK CAPITAL LETTER GAMMA	03	93
Δ	GREEK CAPITAL LETTER DELTA	03	94
$\perp$	UPTACK (ORTHOGONAL TO)	22	A5
$\angle$	ANGLE	22	20

Tableau D.4 (suite)

Caractère	Nom	Rangée	Cellule
⊥	RIGHT ANGLE WITH ARC	22	BE
Θ	GREEK CAPITAL LETTER THETA	03	98
⟨	LEFT POINTING ANGLE BRACKET (BRA)	23	29
⟩	RIGHT POINTING ANGLE BRACKET (KET)	23	2A
Λ	GREEK CAPITAL LETTER LAMBDA	03	9B
'	PRIME	20	32
''	DOUBLE PRIME	20	33
Ξ	GREEK CAPITAL LETTER XI	03	9E
±	MINUS –OR– PLUS SIGN	22	13
Π	GREEK CAPITAL LETTER PI	03	A0
²	SUPERSCRIFT TWO	00	B2
Σ	GREEK CAPITAL LETTER SIGMA	03	A3
×	MULTIPLICATION SIGN	00	D7
³	SUPERSCRIFT THREE	00	B3
Υ	GREEK CAPITAL LETTER UPSILON	03	A5
Φ	GREEK CAPITAL LETTER PHI	03	A6
.	MIDDLE DOT	00	B7
Ψ	GREEK CAPITAL LETTER PSI	03	A8
Ω	GREEK CAPITAL LETTER OMEGA	03	A9
∅	EMPTY SET	22	05

Tableau D.4 (suite)

Caractère	Nom	Rangée	Cellule
→	RIGHTWARDS HARPOON WITH BARB UPWARDS (VECTOR OVERBAR)	21	C0
√	SQUARE ROOT (RADICAL)	22	1A
f	LATIN SMALL LETTER F WITH HOOK (FUNCTION OF)	01	92
∝	PROPORTIONAL TO	22	1D
±	PLUS – MINUS SIGN	00	B1
°	DEGREE SIGN	00	B0
α	GREEK SMALL LETTER ALPHA	03	B1
β	GREEK SMALL LETTER BETA	03	B2
γ	GREEK SMALL LETTER GAMMA	03	B3
δ	GREEK SMALL LETTER DELTA	03	B4
ε	GREEK SMALL LETTER EPSILON	03	B5
ζ	GREEK SMALL LETTER ZETA	03	B6
η	GREEK SMALL LETTER ETA	03	B7
θ	GREEK SMALL LETTER THETA	03	B8
ι	GREEK SMALL LETTER IOTA	03	B9
κ	GREEK SMALL LETTER KAPPA	03	BA
λ	GREEK SMALL LETTER LAMBDA	03	BB
μ	GREEK SMALL LETTER MU	03	BC
ν	GREEK SMALL LETTER NU	03	BD
ξ	GREEK SMALL LETTER XI	03	BE

Tableau D.4 (suite)

Caractère	Nom	Rangée	Cellule
‰	PER MILLE SIGN	20	30
π	GREEK SMALL LETTER PI	03	C0
ρ	GREEK SMALL LETTER RHO	03	C1
σ	GREEK SMALL LETTER SIGMA	03	C3
÷	DIVISION SIGN	03	F7
τ	GREEK SMALL LETTER TAU	03	C4
υ	GREEK SMALL LETTER UPSILON	03	C5
φ	GREEK SMALL LETTER PHI	03	C6
χ	GREEK SMALL LETTER CHI	03	C7
ψ	GREEK SMALL LETTER PSI	03	C8
ω	GREEK SMALL LETTER OMEGA	03	C9
†	DAGGER	20	20
←	LEFTWARDS ARROW	21	90
↑	UPWARDS ARROW	21	91
→	RIGHTWARDS ARROW	21	92
↓	DOWNWARDS ARROW	21	93
—	OVERLINE	20	3E

Bibliographie

- [1] CEI 60027 (toutes les parties), *Symboles littéraux à utiliser en électrotechnique*
- [2] CEI 60748(toutes les parties), *Dispositifs à semiconducteurs – Circuits intégrés*
- [3] CEI 61360-5, *Types normalisés d'éléments de données avec plan de classification pour composants électriques – Partie 5: Extensions au schéma du dictionnaire EXPRESS*
- [4] CEI 80000 (toutes les parties)³, *Grandeurs et unités*
- [5] ISO/CEI 8824-1, *Technologies de l'information – Notation de syntaxe abstraite numéro un (ASN.1): Spécification de la notation de base* (disponible en anglais seulement)
- [6] ISO/CEI 11179-5, *Technologies de l'information – Registres de métadonnées (RM) – Partie 5: Principes de dénomination et d'identification* (disponible en anglais seulement)
- [7] ISO/CEI, *International Classification of Standard (ICS)*
- [8] ISO 31 (toutes les parties), *Grandeurs et unités*
- [9] ISO 639-1, *Codes pour la représentation des noms de langue – Partie 1: Code alpha-2*
- [10] ISO 639-2, *Codes pour la représentation des noms de langue – Partie 2: Code alpha-3*
- [11] ISO 704, *Travail terminologique – Principes et méthodes*
- [12] ISO 843, *Information et documentation – Conversion des caractères grecs en caractères latins*
- [13] ISO 1087-1, *Travaux terminologiques – Vocabulaire – Partie 1: Théorie et application*
- [14] ISO 6523, *Échange de données – Structure pour l'identification des organisations*
- [15] ISO 10303-1:1994, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 1: Aperçu et principes fondamentaux* (disponible en anglais seulement)
- [16] ISO 10303-21, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 21: Méthodes de mise en application: Encodage en texte clair des fichiers d'échange* (disponible en anglais seulement)
- [17] ISO 10303-42:2000, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 42: Ressource générique intégrée: Représentation géométrique et topologique* (disponible en anglais seulement)⁷
- [18] ISO 13584-1:2001, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 1: Aperçu et principes fondamentaux* (disponible en anglais seulement)

⁷ Une nouvelle édition de l'ISO 10303-42 a été publiée en 2003.

- [19] ISO 13584-24:2003, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 24: Ressource logique : Modèle logique de fournisseur* (disponible en anglais seulement)
- [20] ISO 13584-25, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 25: Ressource logique: Modèle logique de fournisseur avec des valeurs d'ensemble et un contenu explicite* (disponible en anglais seulement)
- [21] ISO 13584-32, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 32: Ressources d'implémentation: OntoML: Langage de marquage ontologique* (disponible en anglais seulement)
- [22] ISO 13584-511, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 511: Systèmes mécaniques et composants pour utilisation générale – Dictionnaire de référence pour éléments de fixation* (disponible en anglais seulement)
- [23] ISO/TS 23768-1⁸, *Roulements – Bibliothèque de composants – Partie 1: Dictionnaire de référence des roulements*
- [24] ISO/TS 29002-5, *Systèmes d'automatisation industrielle et intégration – Échange de données caractéristiques – Partie 5: Schéma d'identification* (disponible en anglais seulement)
- [25] ISO/TS 29002-20, *Systèmes d'automatisation industrielle et intégration – Échange de données caractéristiques – Partie 20: Services de résolution d'un dictionnaire de concept* (disponible en anglais seulement)
- [26] ISO 80000 (toutes les parties)⁹, *Grandeurs et unités*
- [27] XML Schema Part 2: Datatypes. Second Edition. World Wide Web Consortium Recommendation 28-October-2004
Disponible sur <<http://www.w3.org/TR/2004/REC-xmlschema-2-20041028>>
- [28] Mathematical Markup Language (MathML). Second Edition. World Wide Web Consortium Recommendation 21-October-2003
Disponible sur <<http://www.w3.org/TR/MathML2/>>

⁸ A publier.

⁹ La série de parties ISO 31 a été annulée et remplacée par la série de parties ISO 80000/CEI 80000.

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
PO Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch